

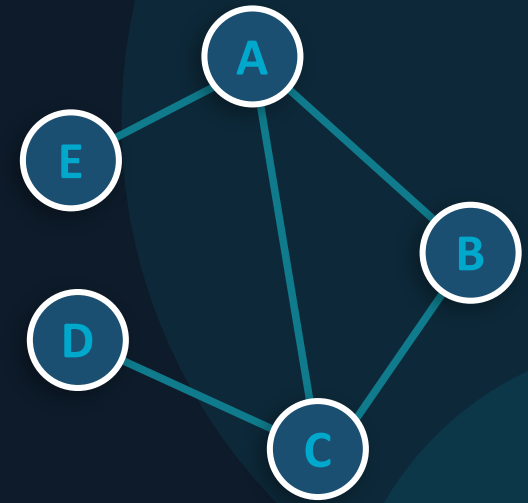
VERİ YAPILARI

Graph (Graf) Veri Yapısı

Temel kavramlar · Türler · Algoritmalar

· Dr. Öğr. Üyesi Abdulsamet AKTAŞ · Dr. Öğr. Üyesi Emrah DİKBİYİK

BFS · DFS · Dijkstra · Prim · Kruskal



İçindekiler

Bu derste neler öğreneceğiz?

01

Graph Nedir?

Tanım, gerçek hayat kullanımları

02

Terminoloji

Vertex, edge, degree, komşuluk

03

Graph Türleri

Yönlü, ağırlıklı, tam, cyle...

04

Yürüyüş Kavramları

Walk, Trail, Path, Cycle, Euler, Hamiltonian

05

Temsil Yöntemleri

Adjacency Matrix, List, Edge List

06

BFS & DFS

Gezinme algoritmaları

07

Dijkstra

En kısa yol algoritması

08

Kruskal, Prim

Minimum Spanning Tree

01

Graph Nedir?

Gerçek hayattaki bağlantıları modelleyen yapı

Graph Nedir?

Matematiksel tanım

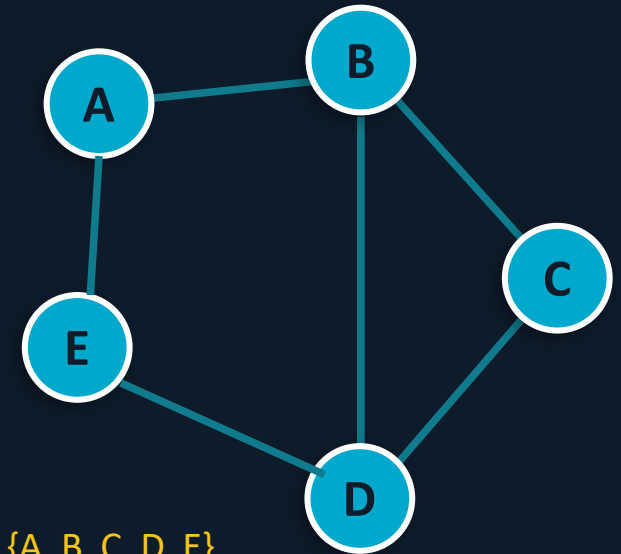
$$G = (V, E)$$

V = Dğümler (Vertices) | E = Kenarlar (Edges)

Tanım:

Graph; düğümler (node/vertex) ve bu düğümler arasındaki ilişkileri temsil eden kenarlardan (edge) oluşan veri yapısıdır. Doğrusal olmayan (non-linear) bir yapıdır ve çok çeşitli gerçek dünya problemlerini modellemek için kullanılır.

$|V| = 5$ düğüm | $|E| = 6$ kenar



$V = \{A, B, C, D, E\}$

$E = \{(A,B), (B,C), (C,D), (D,E), (E,A), (B,D)\}$

Klasik Küme Gösterimi

Gerçek Hayat Örnekleri

Graph her yerde!



Harita & Navigasyon

Şehirler = düğüm

Yollar = kenar

Mesafe = ağırlık

Google Maps, en kısa yolu graph algoritmaları ile bulur!



Sosyal Ağlar

Kullanıcılar = düğüm

Arkadaşlık = kenar

Facebook, Instagram, LinkedIn bağlantıları graph yapısındadır.



İnternet & Web

Web sayfaları = düğüm

Hiperlinkler = kenar

Google PageRank web'i graph olarak modeller.



Paket Bağımlılıkları

Kütüphaneler = düğüm

Bağımlılık = kenar

npm, pip gibi paket yöneticileri graph kullanır.



Elektrik Ağı

Santral/ev = düğüm

Kablolar = kenar

Elektrik dağıtım optimizasyonu graph problemidir.



Biyoinformatik

Proteinler = düğüm

Etkileşim = kenar

Protein etkileşim ağları graph ile analiz edilir.

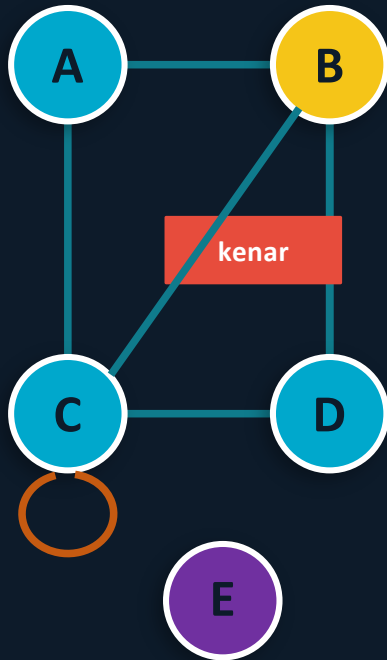
02

Terminoloji

Vertex, Edge, Degree, Komşuluk...

Temel Terimler

Vertex, Edge ve komşuluk



Vertex (Düğüm)

Graphtaki her bir nokta. $V = \{A, B, C, D, E\}$

Edge (Kenar)

İki düğümü birbirine bağlayan ilişki. $(A,B), (B,C), \dots$

Komşu (Neighbor)

Bir kenarla doğrudan bağlı düğümler. B'nin komşuları: A, C, D

Degree

Bir düğüme bağlı kenar sayısı. Örn: $\text{degree}(B) = 3$

Self-loop

Bir düğümden kendine giden kenar

Isolated vertex

Hiçbir kenara sahip olmayan düğüm

03

Graph Türleri

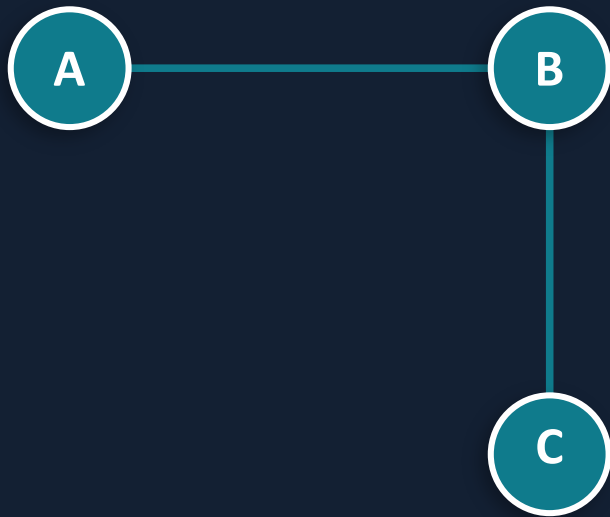
Directed · Undirected · Weighted · Complete · Cyclic...

Yönlü vs Yönsüz Graph

Directed & Undirected

Undirected Graph

Kenarlar iki yönlüdür

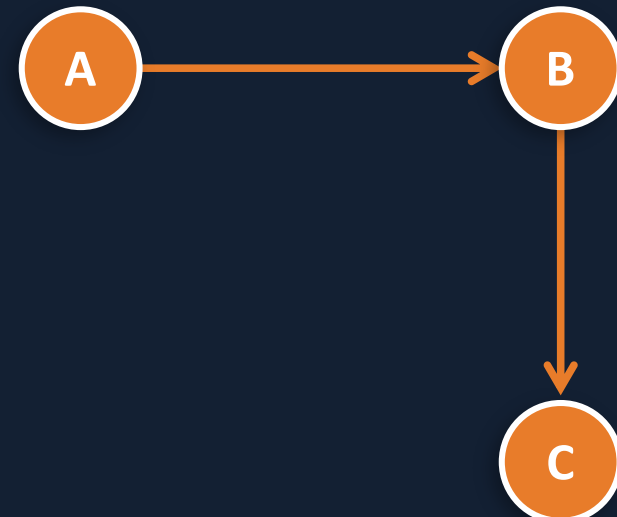


$$(A,B) = (B,A)$$

📌 Sosyal ağ arkadaşlığı (iki yönlü)

Directed Graph (Digraph)

Kenarların yönü vardır



$$(A,B) \neq (B,A)$$

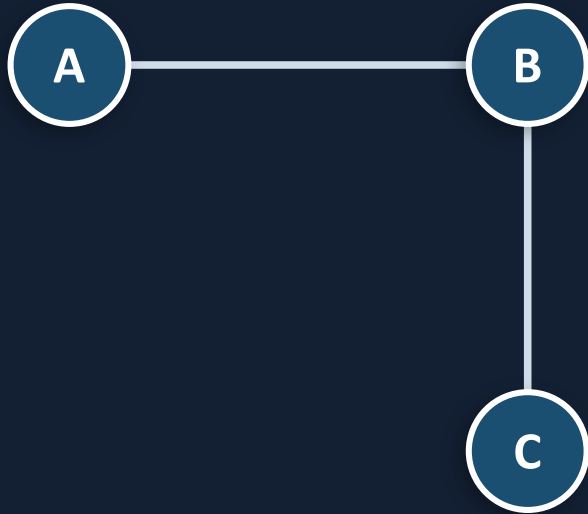
📌 Twitter takip sistemi (tek yönlü)

Ağırlıklı vs Ağırksız Graph

Weighted & Unweighted

Unweighted Graph

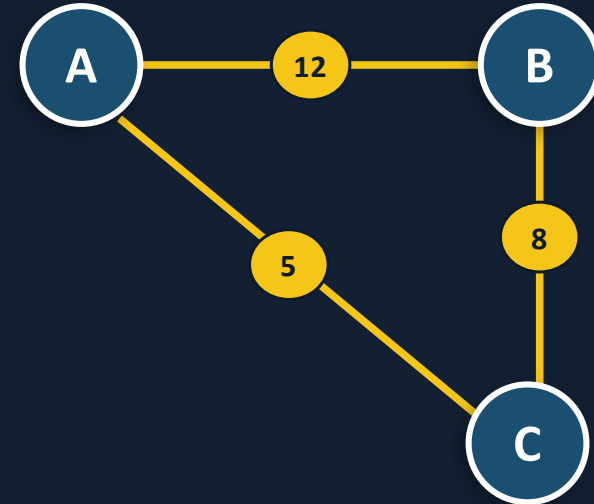
Tüm kenarlar eşit değerde



📌 Facebook arkadaşlık
(sadece bağlı/değil)

Weighted Graph

Her kenara bir değer atanmış

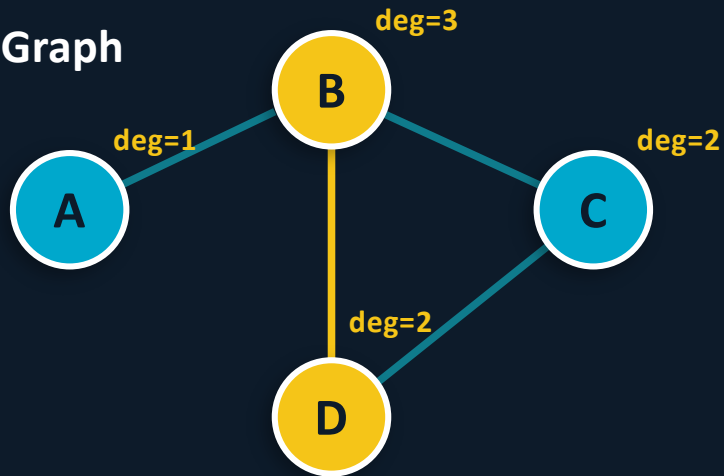


📌 Google Maps mesafe
(km, dakika, trafik)

Degree (Derece)

In-degree ve Out-degree kavramları

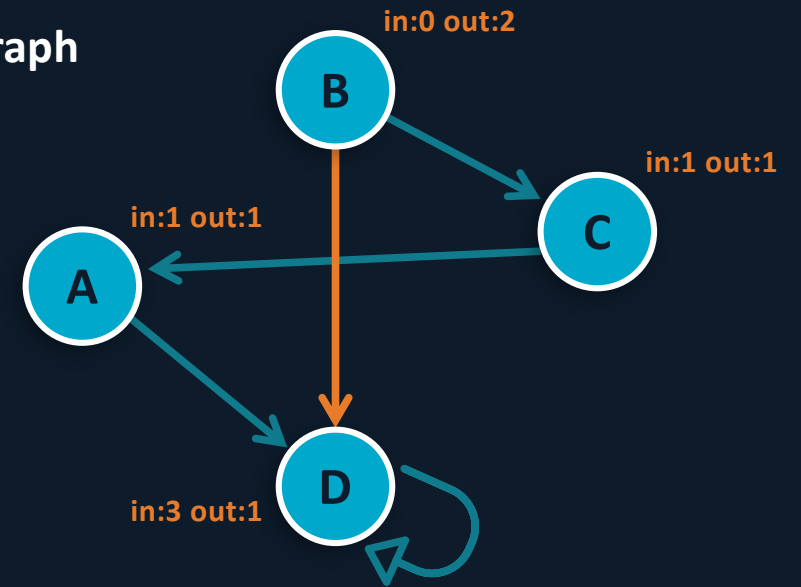
Yönsüz Graph



Kural: $\sum \text{degree} = 2 \times |E|$

Bu örnekte: $1+3+2+2 = 8 = 2 \times 4 = 8$

Yönlü Graph



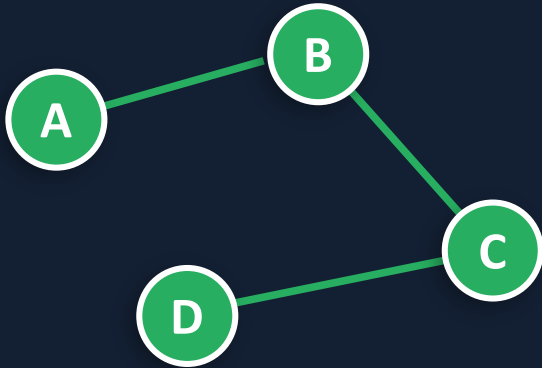
In-degree: Düğüme gelen kenar sayısı

Out-degree: Düğümden çıkan kenar sayısı

Bağlı · Bağlı Değil · Tam Graph

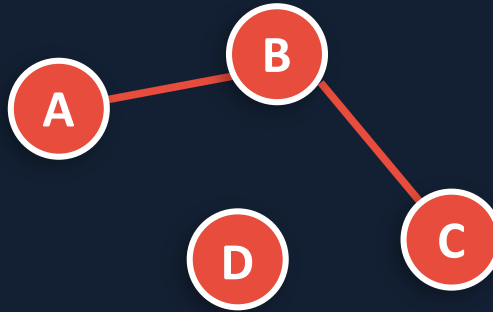
Connected · Disconnected · Complete

Connected



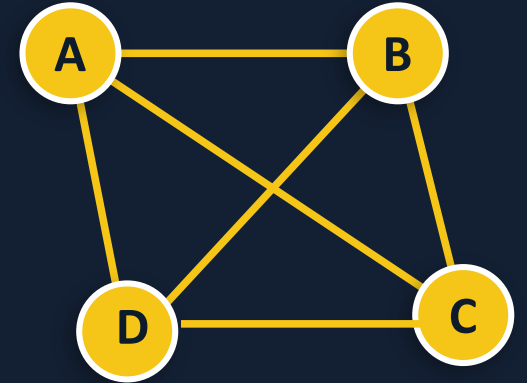
Her düğüme ulaşılabilir

Disconnected



Ulaşılamayan düğümler var

Complete K_4



Her çift düğüm arasında kenar

Tam Graph K_n formülü: $n(n-1)/2$ kenar

K_4 : $4 \times 3 / 2 = 6$ kenar | K_5 : $5 \times 4 / 2 = 10$ kenar | K_{10} : $10 \times 9 / 2 = 45$ kenar

Cyclic Graph

En az bir döngü (cycle) içerir

Acyclic Graph (DAG)

Hiç döngü içermez. DAG = Directed Acyclic Graph

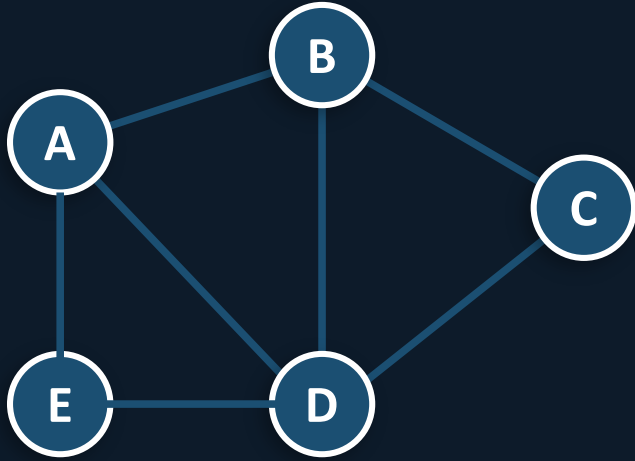
04

Yürüyüş Kavramları

Walk · Trail · Path · Cycle · Euler · Hamiltonian

Walk · Trail · Path

Graphta yürüyüş kavramları



Referans Graph

$\text{Walk} \supseteq \text{Trail} \supseteq \text{Path}$ (Her path bir trail, her trail bir walk'tur)

Walk (Yürüyüş)

Kural: Düğümler VE kenarlar tekrarlanabilir

Örnek: $A \rightarrow B \rightarrow D \rightarrow A \rightarrow E \rightarrow D$

💡 En genel senaryo

Trail (İz)

Kural: Kenarlar tekrarlanamaz, düğümler tekrarlanabilir

Örnek: $A \rightarrow B \rightarrow D \rightarrow E \rightarrow A \rightarrow (B \text{ değil, } A-B \text{ kullanıldı})$

💡 Euler path bu kategoridedir

Path (Yol)

Kural: Düğümler VE kenarlar tekrarlanamaz

Örnek: $A \rightarrow B \rightarrow C \rightarrow D$

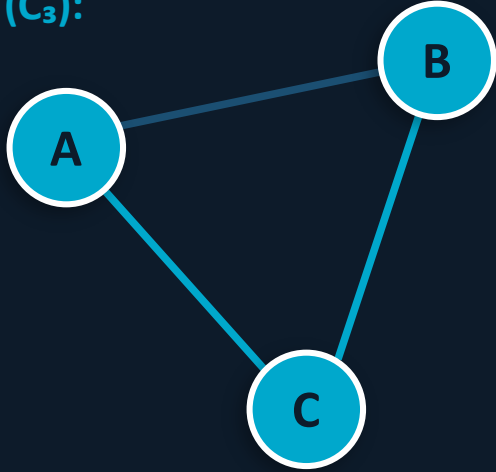
💡 En kısıtlı yürüyüş

Cycle (Döngü)

Başladığı düğüme dönen path

Tanım: Başlangıç ve bitiş düğümü aynı olan, diğer düğümlerin tekrarlanmadığı kapalı path.

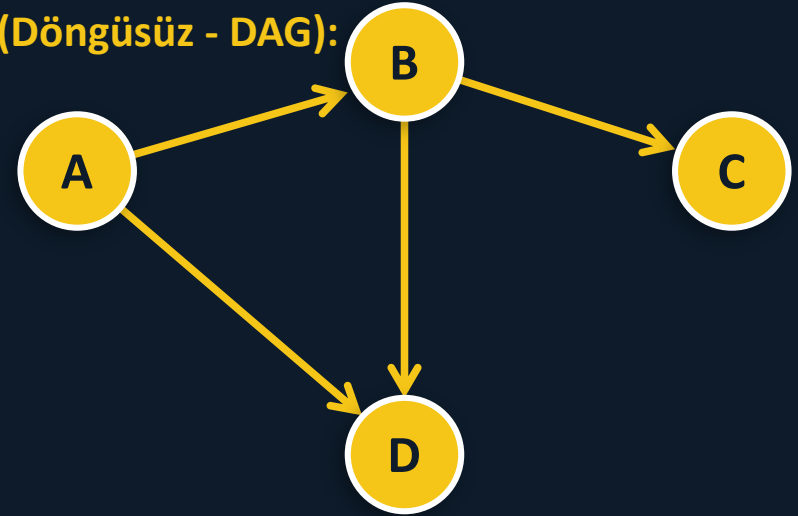
Cycle var (C_3):



Döngü: $A \rightarrow B \rightarrow C \rightarrow A$

Uzunluk = 3 (C_3 = üçgen)

Acyclic (Döngüsüz - DAG):



Hiçbir düğüme geri dönülemez

Bağımlılık grafları, iş akışları bu yapıdadır

Cycle tespiti: DFS sırasında ziyaret edilmiş bir düğüme tekrar ulaşırsa döngü var demektir.

Euler Path & Circuit

Tüm kenarları tam bir kez geçen yol

Euler Path:

Tüm kenarları tam bir kez kullanan yol.
Başlangıç $\neq$ Bitiş

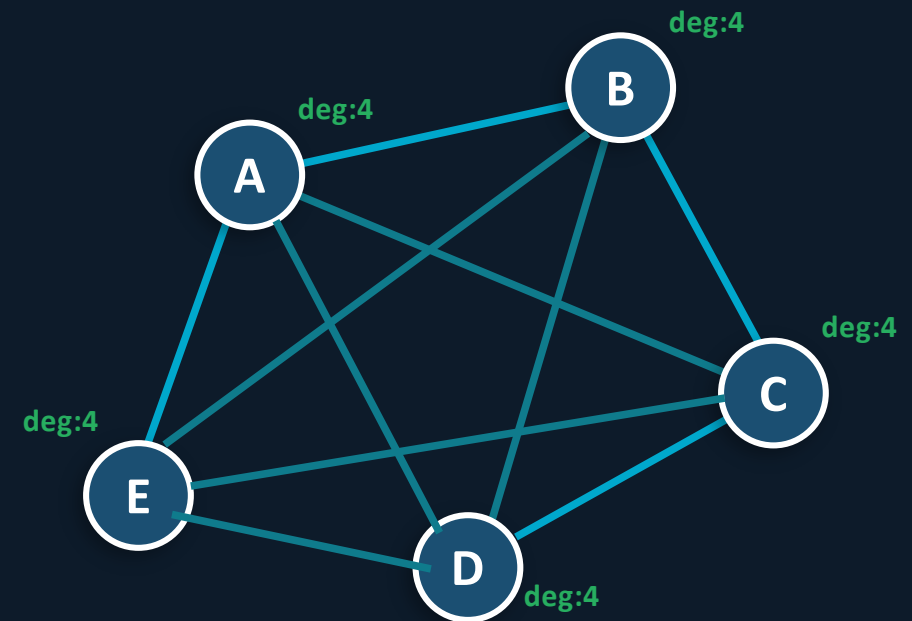
Euler Path Koşulu:

- ✓ Tam bağlı graph
- ✓ Tam olarak 2 düğümün derecesi tek sayı olabilir.

Euler Circuit:

Tüm kenarları tam bir kez kullanan KAPALI yol.
Başlangıç = Bitiş

- ✓ **Euler Circuit:** TÜM düğümlerin derecesi çift sayı



Tüm dereceler çift $\rightarrow$ Euler Circuit VAR!

Bir yol: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow A \rightarrow C \rightarrow \dots$

Hamiltonian Path & Circuit

Tüm düğümleri tam bir kez ziyaret eden yol

Hamiltonian Path:

Her düğümü tam bir kez ziyaret eden yol.
Tüm kenarları kullanmak zorunlu değil.

Hamiltonian Circuit:

Her düğümü tam bir kez ziyaret eden KAPALI yol.
Başlangıç = Bitiş

Euler vs Hamiltonian

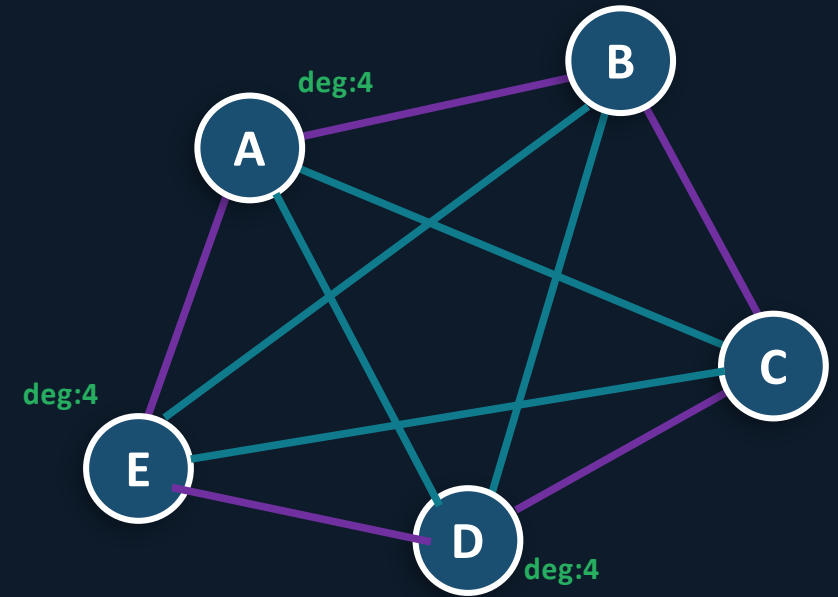
Euler Path: Tüm KENARLAR bir kez

Hamiltonian Path: Tüm DÜĞÜMLER bir kez

⚠ Hamiltonian için kolay formül yok!

NP-complete problemdir

📦 **Hamiltonian Circuit = Gezgin Satıcı Problemi (TSP)!**



Hamiltonian Circuit (mor): A→B→C→D→E→A

Mavi kenarlar kullanılmayabilir

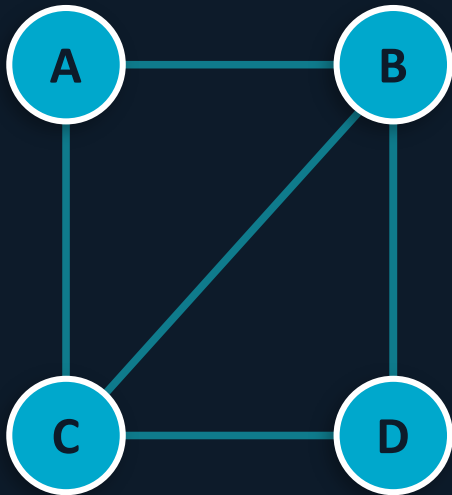
05

Temsil Yöntemleri

Adjacency Matrix · Adjacency List · Edge List

Adjacency Matrix

Komşuluk Matrisi



	A	B	C	D
A	0	1	1	0
B	1	0	1	1
C	1	1	0	1
D	0	1	1	0

$M[i][j] = 1 \rightarrow i$ ve j arasında kenar var

Undirected $\rightarrow$ Matris simetrik ($M[i][j] = M[j][i]$)

✓ Avantaj

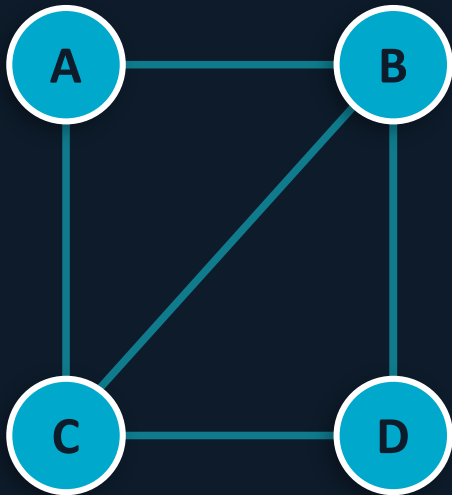
$O(1)$ kenar kontrolü

✗ Dezavantaj

$O(V^2)$ bellek

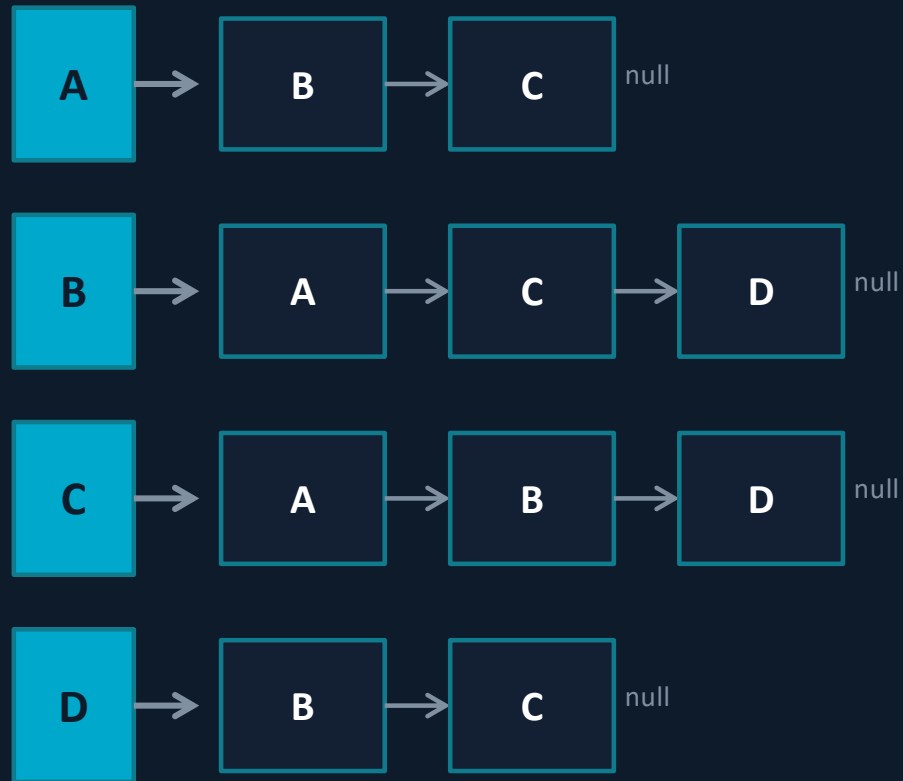
Adjacency List

Komşuluk Listesi



✓ Seyrek graphlarda verimli: $O(V+E)$ bellek

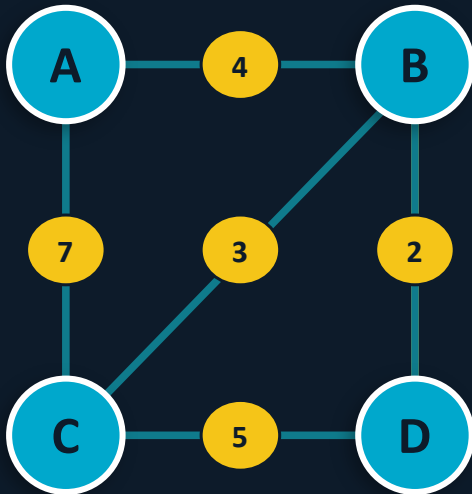
✗ Kenar kontrolü $O(V)$ - yavaş olabilir



```
graph = {'A': ['B', 'C'], 'B': ['A', 'C', 'D'], 'C': ['A', 'B', 'D'], 'D': ['B', 'C']}
```

Ağırlıklı Graph — Adjacency Matrix

Kenar ağırlıkları matrise nasıl yansır?



Yönsüz Ağırlıklı Graph

Kenar yok $\rightarrow 0$
Kenar var $\rightarrow$ ağırlık değeri

Adjacency Matrix (Ağırlıklı):

	A	B	C	D
A	0	4	7	0
B	4	0	3	2
C	7	3	0	5
D	0	2	5	0

$M[i][j] = 0 \rightarrow i$ ve j arasında kenar yok

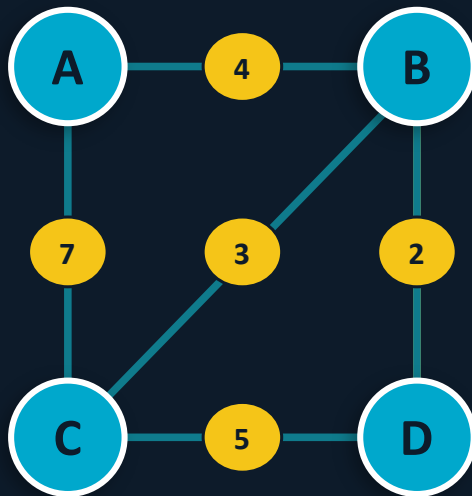
$M[i][j] = w \rightarrow$ kenar var, ağırlığı w

Matris simetriktir (yönsüz)

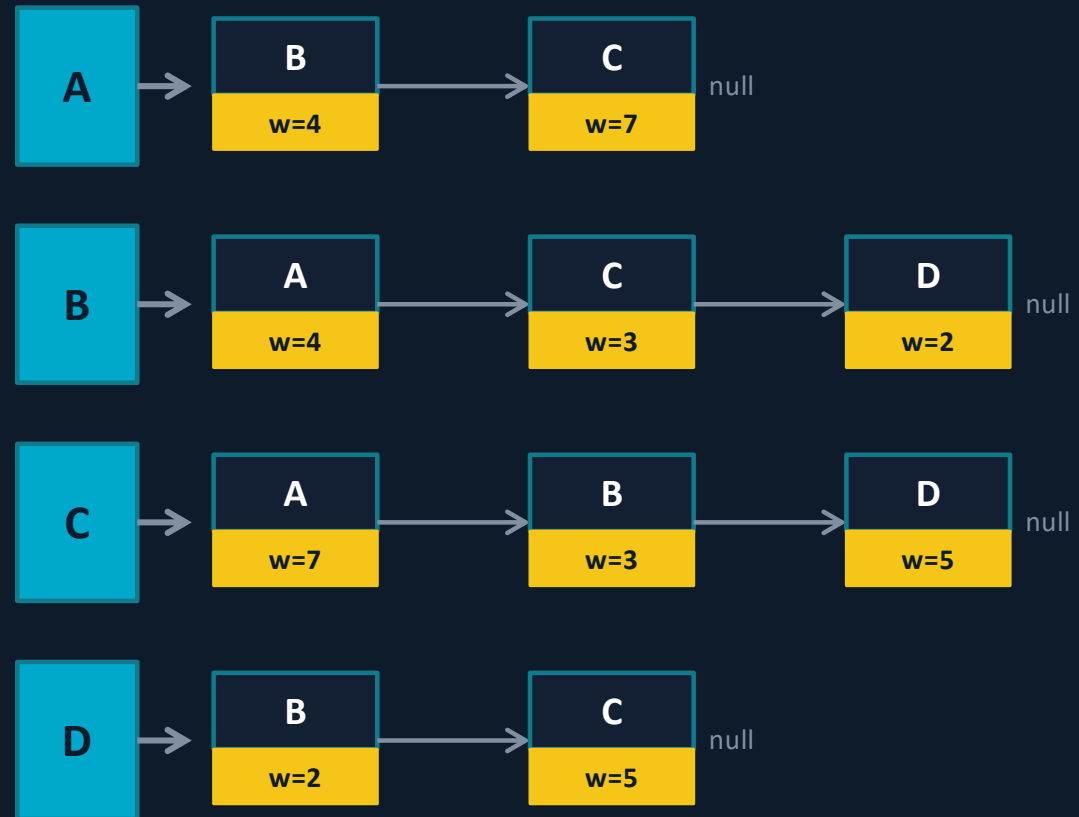
Köşegen = 0 (self-loop yok)

Ağırlıklı Graph — Adjacency List

Her komşuya ağırlık da eklenir



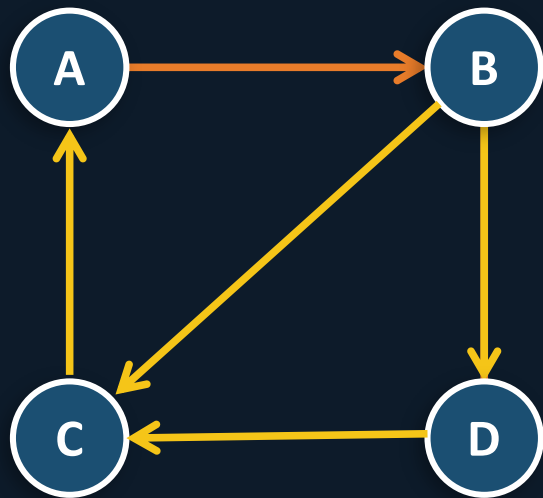
Adjacency List (Ağırlıklı):



```
graph = {'A': [('B',4),('C',7)], 'B': [('A',4),('C',3),('D',2)],  
...}
```

Yönlü Graph — Adjacency Matrix

$M[i][j] \neq M[j][i]$ — matris asimetriktir!



Kenarlar:

- $A \rightarrow B$
- $B \rightarrow D$
- $C \rightarrow A$
- $D \rightarrow C$
- $B \rightarrow C$

Adjacency Matrix (Yönlü):

	A	B	C	D
A	0	1	0	0
B	0	0	1	1
C	1	0	0	0
D	0	0	1	0

Satır = Kaynak (From)

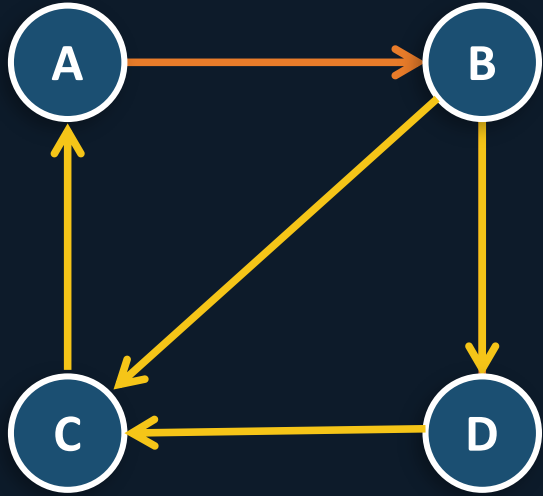
Sütun = Hedef (To)

⚠ Asimetri!

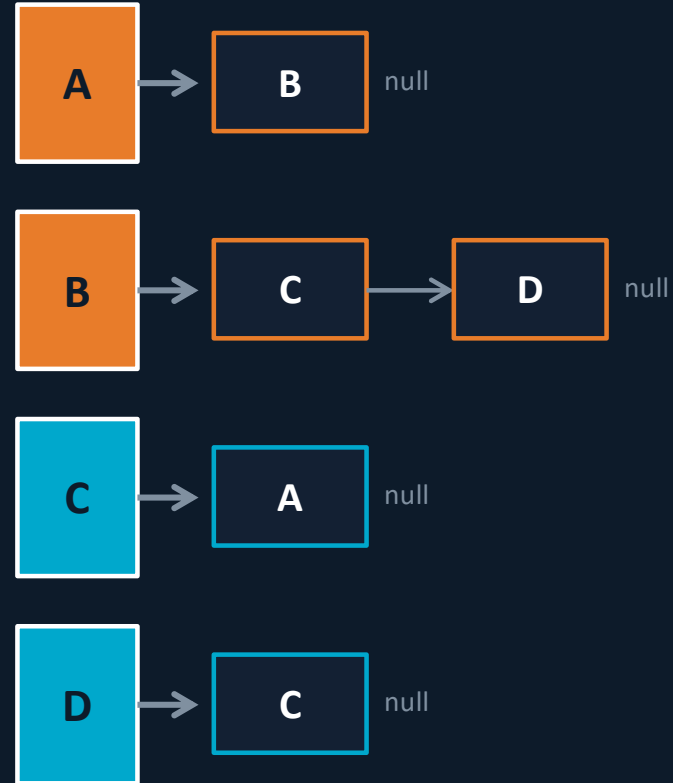
$M[A][B]=1$ ($A \rightarrow B$ var)
 $M[B][A]=0$ ($B \rightarrow A$ yok)

Yönlü Graph — Adjacency List

Listede sadece giden kenarlar tutulur



Adjacency List (Yönlü):



Yönsüz vs Yönlü Liste:

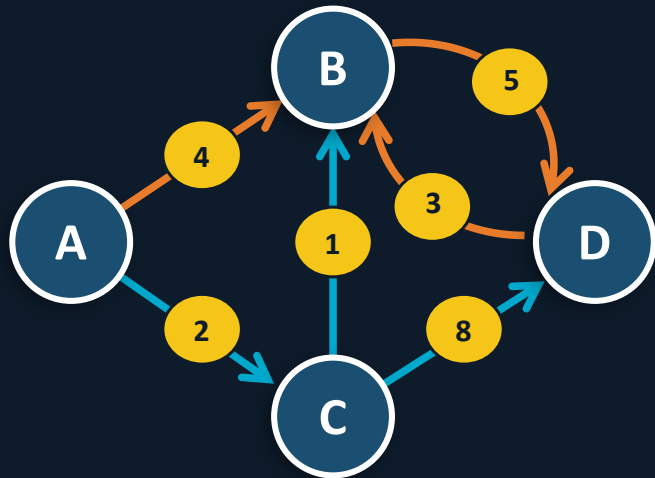
Yönsüz: $A \rightarrow B$ ve $B \rightarrow A$ ikisi de listede

Yönlü: Sadece giden yön listede

Yönlü Adj. List: $O(V + E)$ | Her kenar listede yalnızca 1 kez görünür

Yönlü + Ağırlıklı Graph — Adjacency Matrix

$M[i][j]$ = ağırlık | $M[j][i]$ genellikle 0 (asimetrik)



Kenarlar:

- A→B (4)
- A→C (2)
- B→D (5)
- C→B (1)
- C→D (8)
- D→B (3)

Adjacency Matrix (Yönlü + Ağırlıklı):

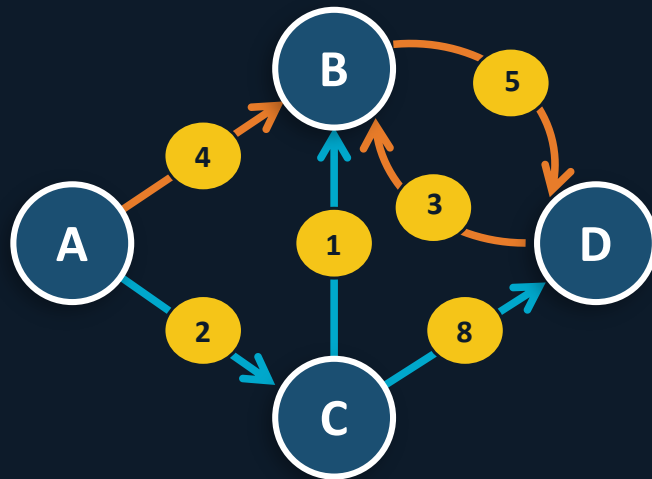
	A	B	C	D
A	0	4	2	0
B	0	0	0	5
C	0	1	0	8
D	0	3	0	0

⚠ Asimetri Örneği:

$M[A][B] = 4$ → A'dan B'ye kenar VAR, ağırlık 4
 $M[B][A] = 0$ → B'den A'ya kenar YOK

Yönlü + Ağırlıklı Graph — Adjacency List

Her eleman (hedef_düğüm, ağırlık) çiftidir



Neden sadece giden kenarlar?

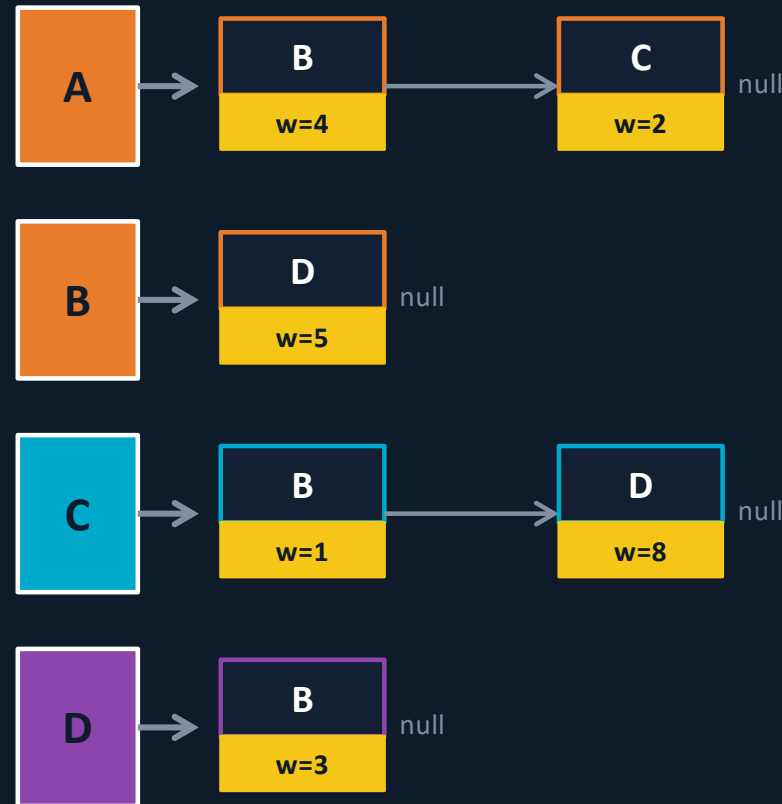
B'nin listesi: [(D,5)]

→ B'den sadece D'ye gidebilirsin

B'ye gelenler (A, C, D) listede YOK

→ In-degree için ters liste gerekir

Adjacency List (Yönlü + Ağırlıklı):



```
graph = {'A': [('B', 4), ('C', 2)], 'B': [('D', 5)], 'C': [('B', 1), ('D', 8)], 'D': [('B', 3)]}
```

Temsil Yöntemleri — Tam Karşılaştırma

4 senaryo için hangi yapı nasıl değişir?

Senaryo	Adj. Matrix'te $M[i][j]$	Adj. List'te Eleman
Yönsüz + Ağırlıksız	$M[i][j] \in \{0, 1\}$ Simetrik	[B, C, D] (sadece düğüm adı)
Yönsüz + Ağırlıklı	$M[i][j] \in \{0, w\}$ Simetrik	[(B,4), (C,7)] (düğüm adı + ağırlık)
Yönlü + Ağırlıksız	$M[i][j] \in \{0, 1\}$ Asimetrik olabilir	[B, C] (sadece giden düğümler)
Yönlü + Ağırlıklı	$M[i][j] \in \{0, w\}$ Asimetrik olabilir	[(B,4), (C,3)] (giden düğüm + ağırlık)

Hangi temsil ne zaman?

- Matrix: Hızlı kenar kontrolü gerekiyorsa ($O(1)$) → Yoğun graph
- Ağırlıklı + Yönlü: Dijkstra için → Adj. List + Priority Queue
- List: Bellek verimliliği önemliyse → Seyrek graph, BFS/DFS
- MST (Kruskal): Edge List kullan → Kenarları sıralamak kolaylaşır

Edge List & Karşılaştırma

Üç temsil yönteminin karşılaştırması

Edge List

Her kenarı (u, v) çifti olarak sakla:

(A, B)

(A, C)

(B, C)

(B, D)

(C, D)

Kruskal algoritması edge list kullanır!

	Adj. Matrix	Adj. List	Edge List
Bellek	$O(V^2)$	$O(V+E)$	$O(E)$
Kenar Ekle	$O(1)$	$O(1)$	$O(1)$
Kenar Sil	$O(1)$	$O(V)$	$O(E)$
Kenar Var mı?	$O(1)$	$O(V)$	$O(E)$
Komşular	$O(V)$	$O(\text{deg})$	$O(E)$
Dense Graph?	✓ İyi	✗ Kötü	✗ Kötü
Sparse Graph?	✗ Kötü	✓ İyi	✓ İyi

06

BFS & DFS

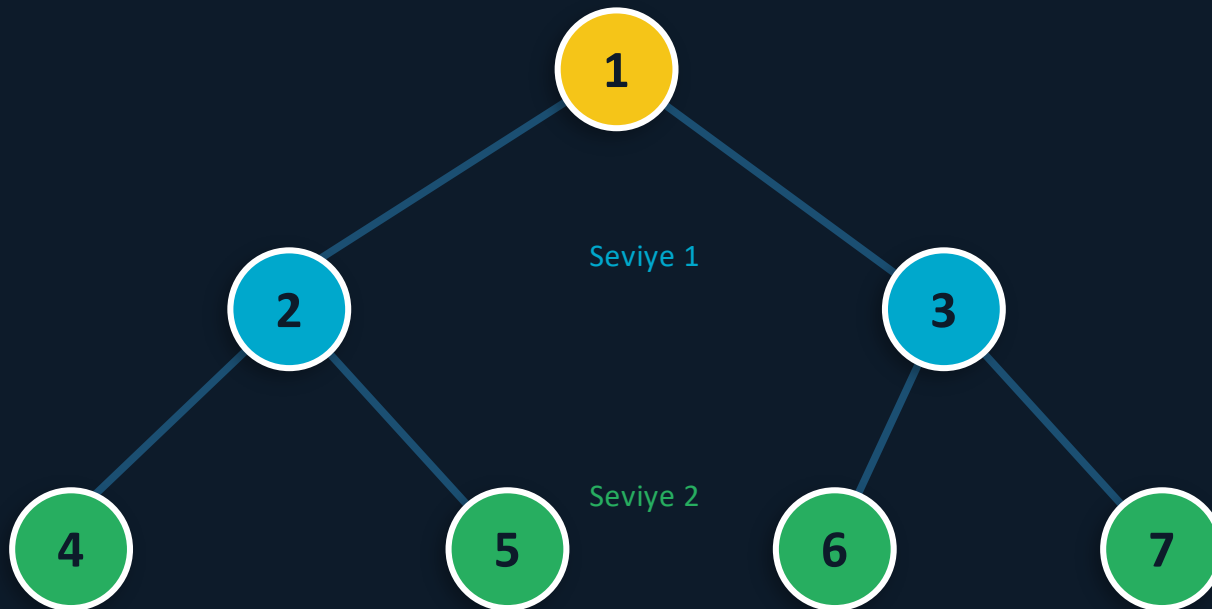
Graph Gezinme Algoritmaları

BFS — Breadth-First Search

Genişlik Öncelikli Arama

Temel Fikir: Katman katman genişle. Önce tüm komşuları ziyaret et, sonra onların komşularına geç.

Veri Yapısı: Queue (Kuyruk) — FIFO

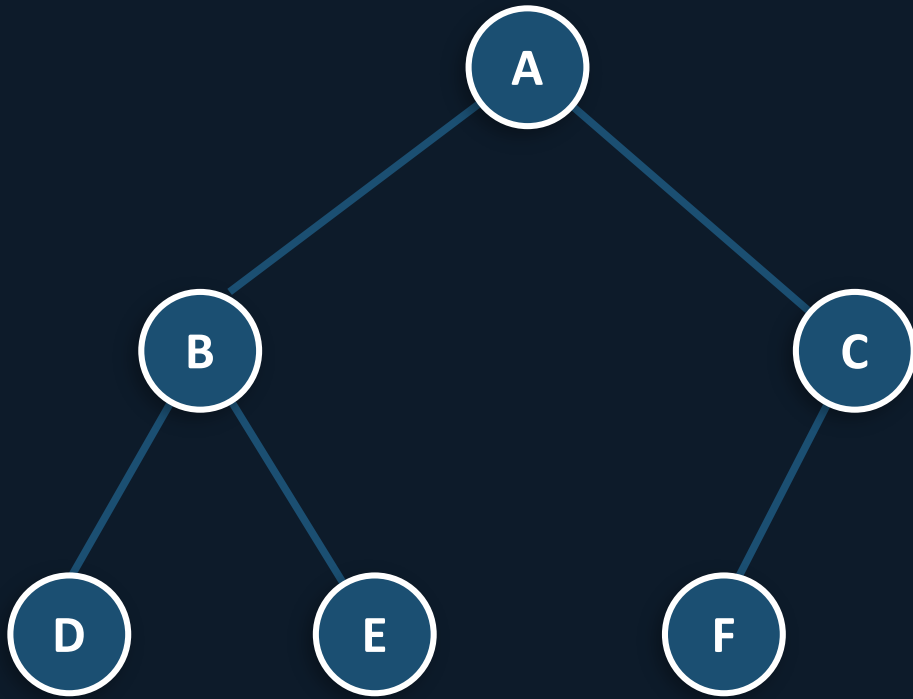


BFS Sırası: 1 → 2 → 3 → 4 → 5 → 6 → 7

Queue: [1] → dequeue(1), enqueue(2,3) → [2,3] → dequeue(2), enqueue(4,5) → [3,4,5] → dequeue(3), enqueue(6,7) → ...

BFS — Adım Adım Örnek

A'dan başlayarak tüm düğümleri ziyaret et



BFS Ziyaret Sırası: A → B → C → D → E → F

Adım 1

Q: [A]

A'yı kuyruğa ekle, başlat

Adım 2

Q: [B, C]

A'yı çıkar → komşu B,C'yi ekle

Adım 3

Q: [C, D, E]

B'yı çıkar → komşu D,E'yi ekle

Adım 4

Q: [D, E, F]

C'yı çıkar → komşu F'yi ekle

Adım 5

Q: [E, F]

D'yı çıkar → komşu yok

Adım 6

Q: [F]

E'yı çıkar → komşu yok

Adım 7

Q: []

F'yı çıkar → Bitti!

BFS — Pseudocode & Kullanım

Algoritma ve uygulamalar

BFS Pseudocode

```
BFS(graph, start):  
    visited = set()  
    queue = Queue()  
    queue.enqueue(start)  
    visited.add(start)  
  
    while queue not empty:  
        node = queue.dequeue()  
        process(node)  
        for neighbor in graph[node]:  
            if neighbor not in visited:  
                visited.add(neighbor)  
                queue.enqueue(neighbor)
```

Zaman: $O(V + E)$ | Alan: $O(V)$

En Kısa Yol

Ağırlıksız graphlarda iki nokta arasındaki en kısa yolu bulur

Web Tarama

Arama motorları web sayfalarını BFS ile tarar (crawler)

Sosyal Ağ

Facebook'ta 'N derece uzaktaki arkadaşlar' hesaplaması

Bileşen Bulma

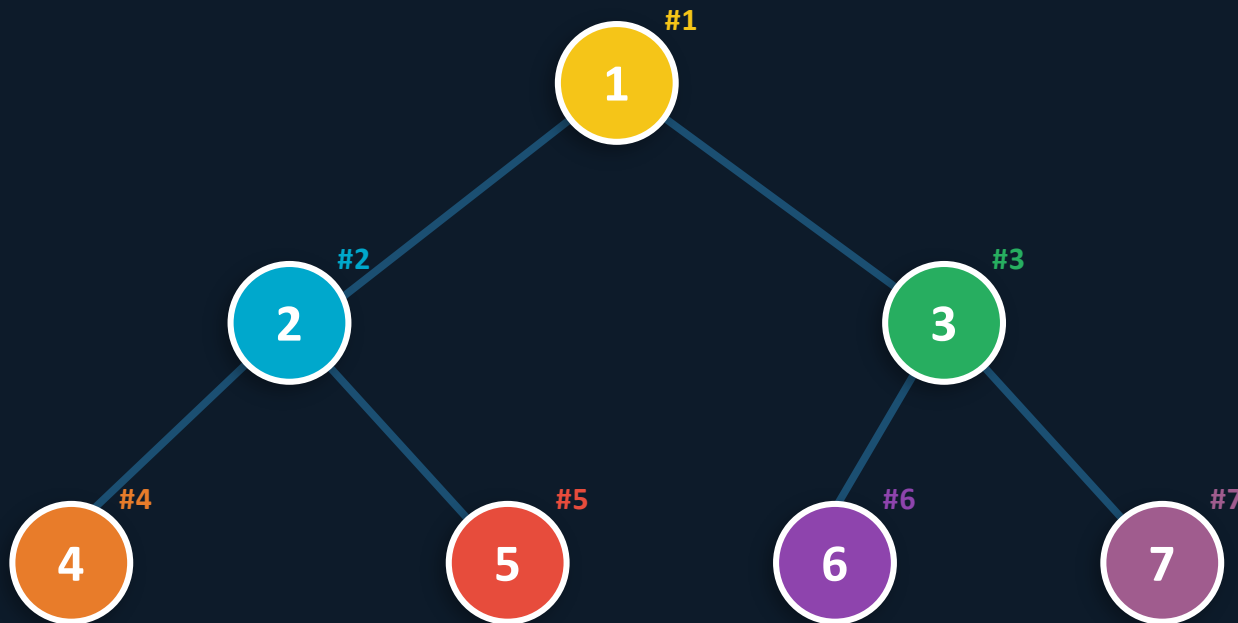
Bağlı bileşenleri (connected components) tespit eder

DFS — Depth-First Search

Derinlik Öncelikli Arama

Temel Fikir: Mümkün olduğunca derine git. Çıkmaz sokak bulunca geri dön (backtrack).

Veri Yapısı: Stack (Yığın) ya da Rekürsiyon

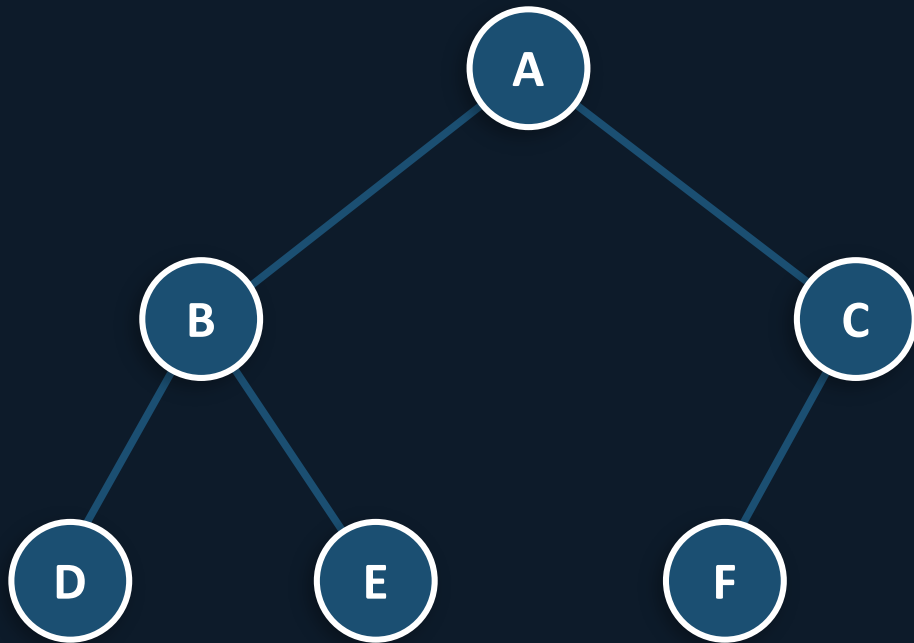


DFS Sırası: 1 → 2 → 4 → 5 → 3 → 6 → 7

Stack: push(1) → pop(1), push(3,2) → pop(2), push(5,4) → pop(4), pop(5) → pop(3), push(7,6) → pop(6) → pop(7)

DFS — Adım Adım Örnek

A'dan başlayarak derinlemesine git



DFS Ziyaret Sırası: $A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F$

BFS sırası: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F$

DFS sırası: $A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F$

Adım 1

S: [A]

A'yı stack'e ekle

Adım 2

S: [C, B]

A'yı pop et, B,C push et

Adım 3

S: [C, E, D]

B'yı pop et, D,E push et

Adım 4

S: [C, E]

D'yı pop et, komşu yok

Adım 5

S: [C]

E'yı pop et, komşu yok

Adım 6

S: [F]

C'yı pop et, F push et

Adım 7

S: []

F'yı pop et → Bitti!

DFS — Pseudocode & Kullanım

Algoritma ve uygulamalar

DFS Pseudocode (Rekürsif)

```
visited = set()
```

```
DFS(graph, node):
```

```
    visited.add(node)
```

```
    process(node)
```

```
    for neighbor in graph[node]:
```

```
        if neighbor not in visited:
```

```
            DFS(graph, neighbor)
```

```
# Başlatma:
```

```
DFS(graph, start_node)
```

Zaman: $O(V + E)$ | Alan: $O(V)$ — call stack nedeniyle

Döngü Tespiti

DFS ile zaten ziyaret edilmiş bir düğüme tekrar erişilirse döngü var

Topological Sort

DAG'larda bağımlılık sıralaması (görev planlama)

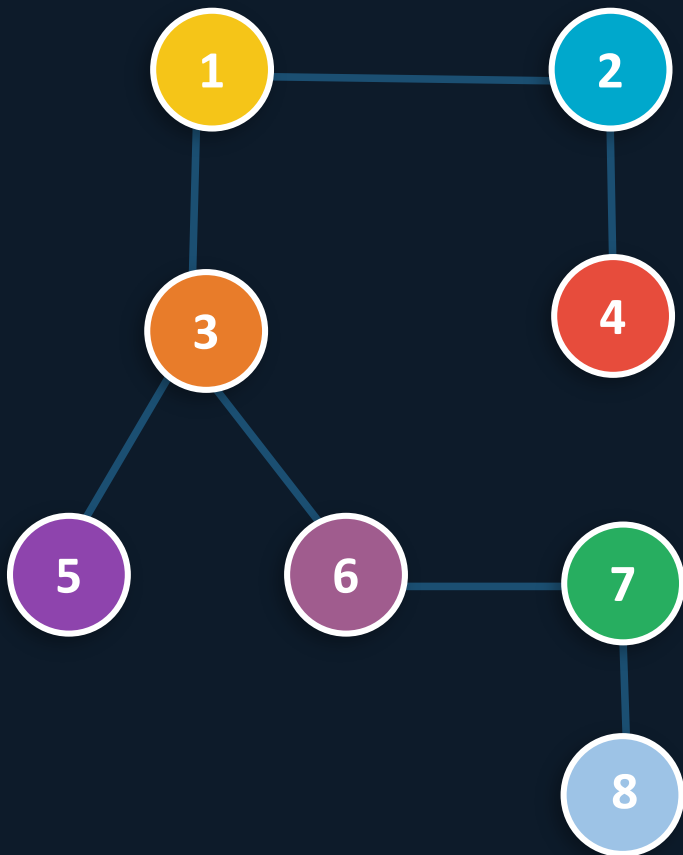
Bağlantı Bileşeni

Birbirinden kopuk bileşenleri tespit eder

Labirent Çözme

Tüm olası yolları keşfeder, backtracking ile çözer

BFS- Örnek-Kuyruk



Adım

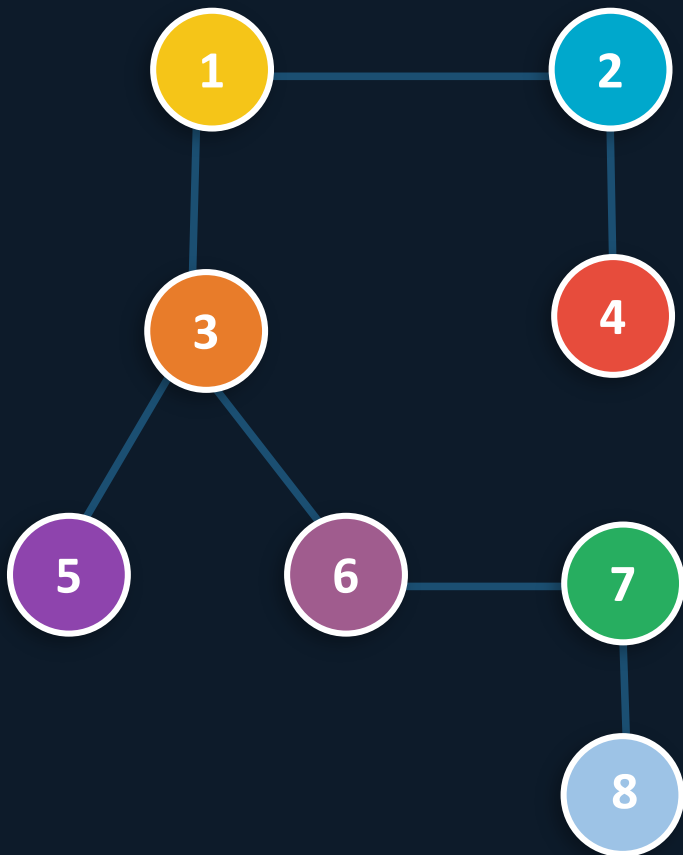
Eklenen

Çıkan

Durum

Output

DFS- Örnek Stack



Adım

Stack Durumu

Çıkan

Output

BFS vs DFS Karşılaştırma

Hangi algoritma ne zaman kullanılır?

Özellik	BFS	DFS
Veri Yapısı	Queue (Kuyruk)	Stack / Rekürsiyon
Gezinme	Seviye seviye	Dal dal (derine)
En Kısa Yol	✓ Ağırlıksız graph	✗ Garanti etmez
Bellek	$O(V)$ — daha fazla	$O(h)$ — h =derinlik
Döngü Tespiti	✓ Evet	✓ Evet
Topo. Sort	✗ Hayır	✓ Evet

🎯 BFS: En kısa yol, seviye analizi

🎯 DFS: Döngü, topoloji, labirent

07

Dijkstra Algoritması

Ağırlıklı graphlarda en kısa yol

Dijkstra — Temel Fikir

En kısa yolu bul

Amaç:

Ağırlıklı bir graptta kaynak düğümden tüm diğer düğümlere olan en kısa yolları bul.

Strateji:

Greedy (Açgözlü) yaklaşım. Her adımda en yakın ziyaret edilmemiş düğümü seç.

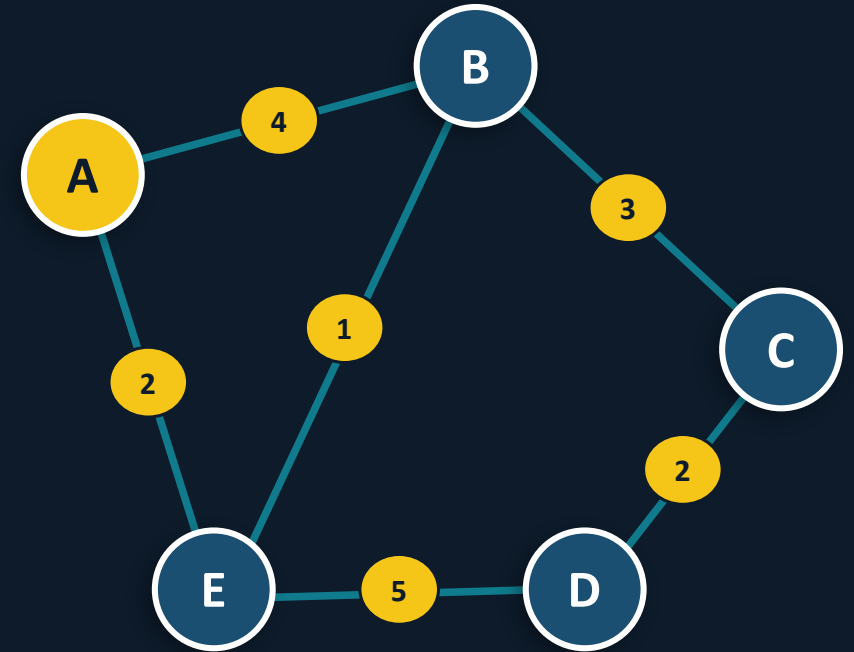
Koşul: Negatif ağırlıklı kenar olmamalı!

$\text{dist}[s] = 0$ (kaynak)

$\text{dist}[v] = \infty$ (diğerleri)

Priority Queue ile en yakın düğümü seç

Zaman: $O((V + E) \log V)$ — Priority Queue ile

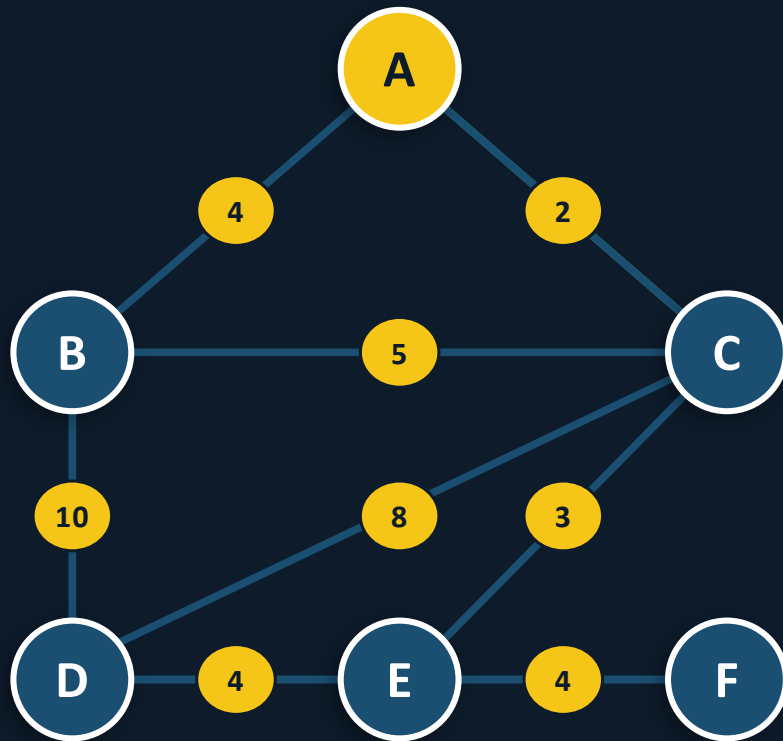


Kaynak: A

En kısa mesafeler hesaplanacak...

Dijkstra — Örnek Graph

A'dan tüm düğümlere en kısa yol



Başlangıç Mesafeleri:

A dist = 0 (kaynak)

B dist = ∞

C dist = ∞

D dist = ∞

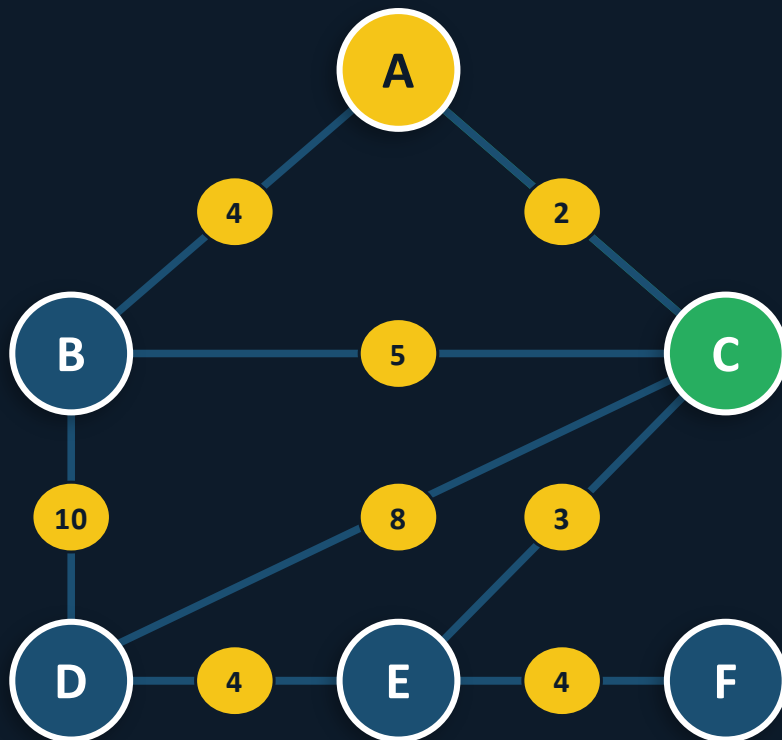
E dist = ∞

F dist = ∞

Sonraki slaytlarda adım adım izleyeceğiz...

Dijkstra — Adım Adım (1/2)

$A \rightarrow B=4$, $A \rightarrow C=2 \rightarrow C$ daha yakın!



Adım 1

A seç (dist=0). A'nın komşuları:

B: $\min(\infty, 0+4)=4$

C: $\min(\infty, 0+2)=2$

Adım 2

C seç (dist=2, en küçük). C'nin komşuları:

B: $\min(4, 2+5)=4$

E: $\min(\infty, 2+3)=5$

D: $\min(\infty, 2+8)=10$

Adım 3

B seç (dist=4). B'nin komşuları:

D: $\min(10, 4+10)=10$

(B→C zaten ziyaret)

Mevcut Mesafeler:

A: 0 ✓

D: 10

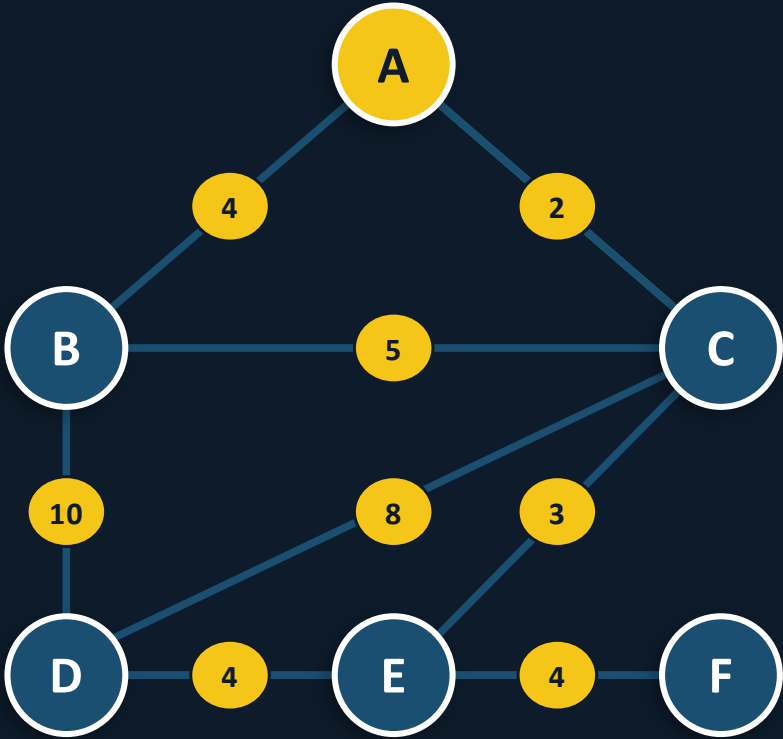
B: 4

E: 5

C: 2 ✓

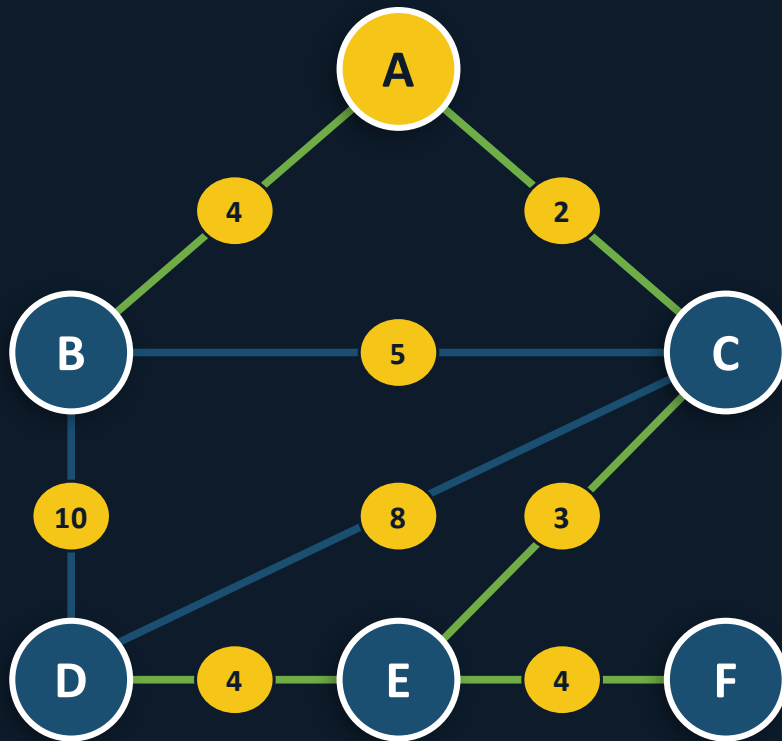
F: ∞

Dijkstra — Adım Adım (1/2)

[illegible]

Dijkstra — Adım Adım (2/2)

Sonuç ve en kısa yollar



Yeşil kenarlar = En Kısa Yol Ağacı (Shortest Path Tree)

En Kısa Yol Sonuçları:

A dist = 0 A

B dist = 4 A → B

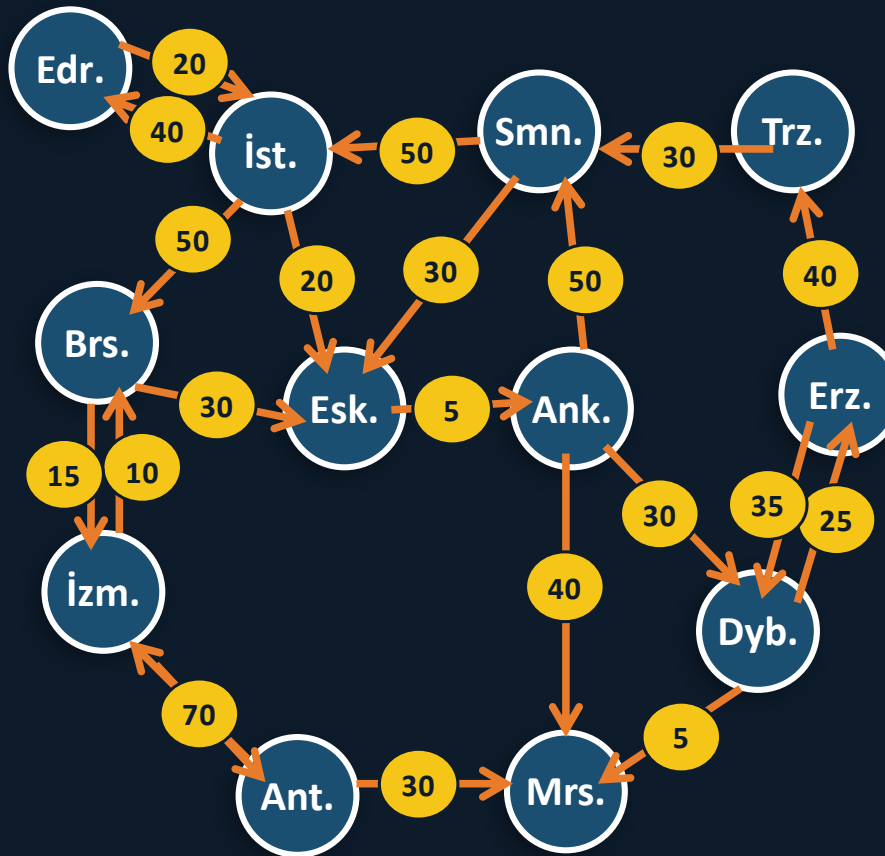
C dist = 2 A → C

D dist = 9 A → C → E → D

E dist = 5 A → C → E

F dist = 9 A → C → E → F

Dijkstra — Örnek-2

[illegible]

Dijkstra — Pseudocode

Öncelikli kuyruk ile implementasyon

Dijkstra(graph, source):

```
dist = {v: ∞ for v in graph}
dist[source] = 0
prev = {v: None for v in graph}
pq = PriorityQueue()
pq.push((0, source))

while pq not empty:
    d, u = pq.pop() # en küçük dist
    if d > dist[u]: continue
    for v, w in graph[u].neighbors:
        alt = dist[u] + w
        if alt < dist[v]:
            dist[v] = alt
            prev[v] = u
            pq.push((alt, v))

return dist, prev
```

Priority Queue

Her zaman en küçük mesafeli düğümü ver

dist[]

Kaynaktan her düğüme bilinen en kısa mesafe

prev[]

En kısa yolu geri izlemek için önceki düğüm

Relaxation

alt < dist[v] kontrolü: yeni yol daha kısa mı?

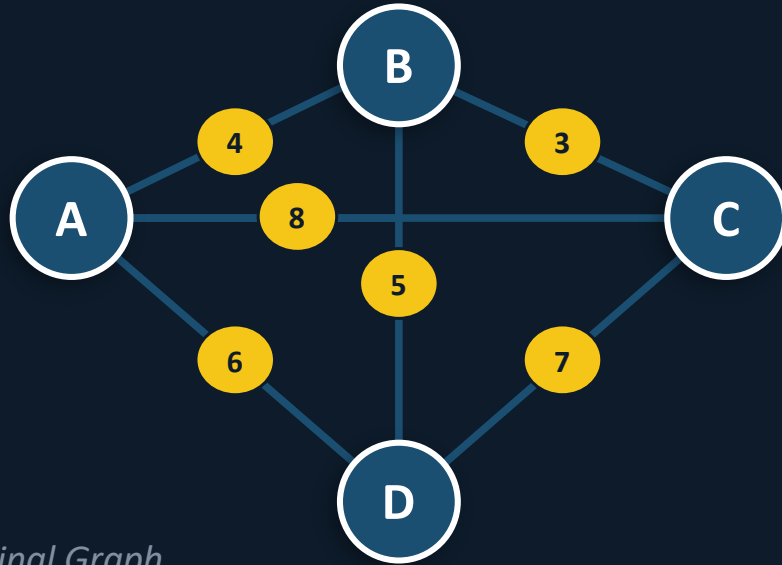
Minimum Spanning Tree (MST) Nedir?

Tüm düğümleri en az maliyetle bağla

Tanım:

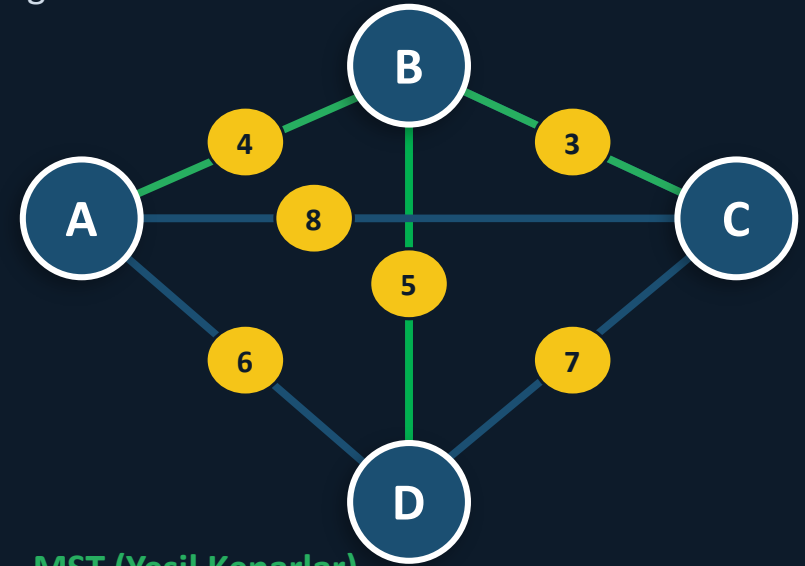
Spanning Tree: Bağlı bir graptaki tüm düğümleri içeren ve hiç döngüsü olmayan alt graph.

Minimum Spanning Tree: Kenar ağırlıklarının toplamı en küçük olan spanning tree.



Original Graph

Tüm kenarlar: 4+6+3+5+7+8=33



MST (Yeşil Kenarlar)

MST toplamı: 3+4+5 = 12 (minimum!)

MST özellikleri: $|V| - 1$ kenar içerir · Döngü yoktur · Bağlantılıdır · Genellikle tekil değildir

08

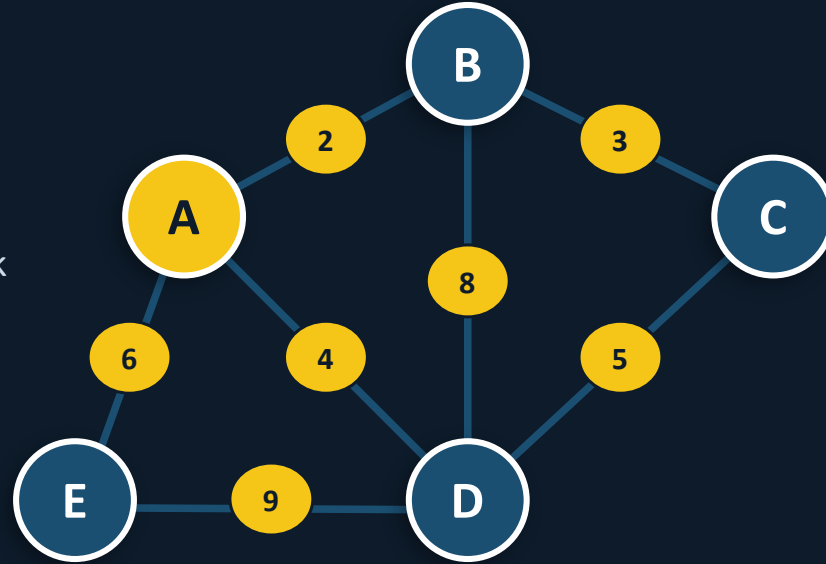
Prim Algoritması

Minimum Spanning Tree — Greedy yaklaşım

Prim — Temel Fikir

MST'yi düğüm ekleyerek büyüt

- 1 Rastgele bir düğümden başla — MST'ye ekle
- 2 MST'deki düğümlere bağlı tüm kesimlere (cut) bak
- 3 Bu kesimler arasından en küçük ağırlıklı kenarı seç
- 4 Seçilen kenarın diğer ucundaki düğümü MST'ye ekle
- 5 Tüm düğümler eklenene kadar 2-4'ü tekrarla



Başlangıç: A

Prim vs Kruskal Farkı

Kruskal: Kenarları sıralar, global en küçüğü seçer

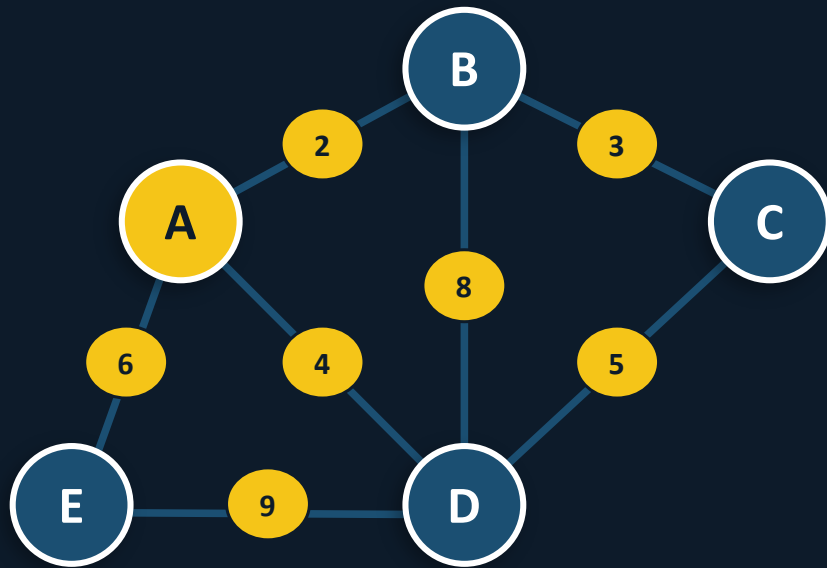
Prim: MST'ye bağlı kesimden lokal en küçüğü seçer

Her ikisi de aynı MST'yi üretir!

Zaman: $O(V^2)$ — basit | $O(E \log V)$ — priority queue ile

Prim — Örnek Graph

A'dan başlayarak MST oluştur



Başlangıç: A (sarı)

Kenarlar (referans):

A-B → $w = 2$ ✓ MST

B-C → $w = 3$ ✓ MST

A-D → $w = 4$ ✓ MST

C-D → $w = 5$

A-E → $w = 6$ ✓ MST

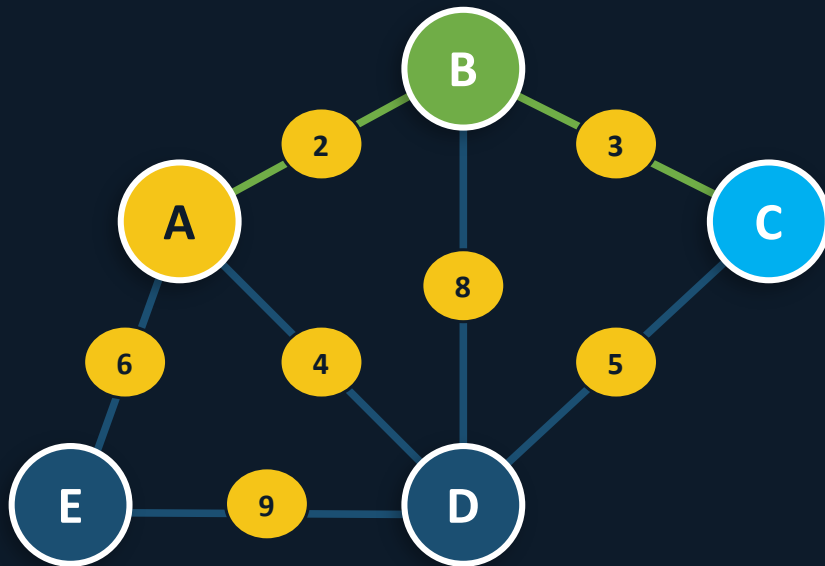
B-D → $w = 8$

D-E → $w = 9$

MST Toplam: $2+3+4+6 = 15$ (5 düğüm, 4 kenar)

Prim — Adım Adım (1/2)

A'dan başla, en yakın komşuyu ekle



MST şu an: {A-B, B-C, A-D} Toplam: 2+3+4 = 9 Kalan: E

Adım 1: Başlangıç

MST = {A}

Kesim kenarları: A-B(2), A-D(4), A-E(6)

En küçük → A-B = 2 → EKLE

Adım 2: B eklendi

MST = {A, B}

Yeni kesim: B-C(3), B-D(8)

Toplam kesim: A-D(4), A-E(6), B-C(3), B-D(8)

En küçük → B-C = 3 → EKLE

Adım 3: C eklendi

MST = {A, B, C}

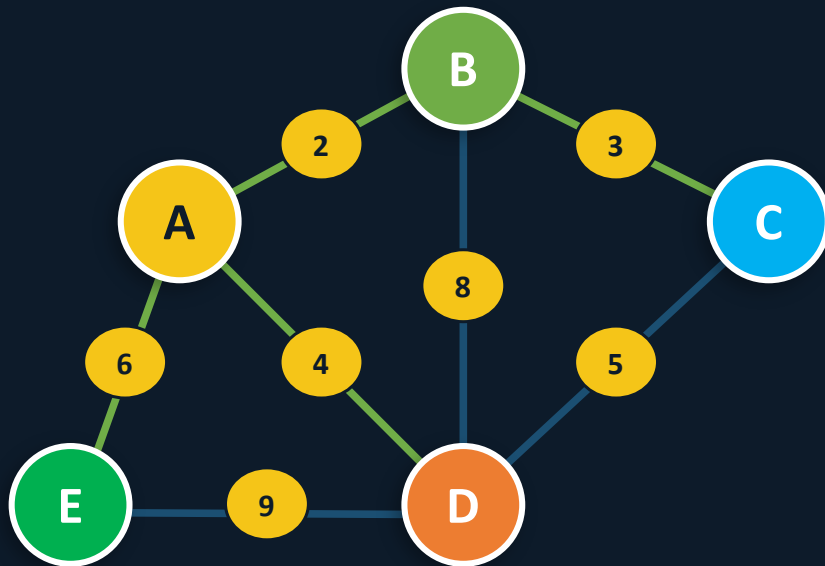
Yeni kesim: C-D(5)

Toplam kesim: A-D(4), A-E(6), B-D(8), C-D(5)

En küçük → A-D = 4 → EKLE

Prim — Adım Adım (2/2)

Tamamla ve sonucu gör



Final MST:

$$A-B(2) + B-C(3) + A-D(4) + A-E(6) = 15$$

Yeşil kenarlar = Minimum Spanning Tree

Adım 4: D eklendi

MST = {A, B, C, D}

Yeni kesim: D-E(9)

Toplam kesim: A-E(6), B-D(8), C-D(5) → zaten içinde, D-E(9)

En küçük (yeni düğüm getirecek) → A-E = 6 → EKLE

Adım 5: E eklendi

MST = {A, B, C, D, E}

Tüm düğümler eklendi → TAMAMLANDI!

Atlanan: C-D(5) zaten bağlı, B-D(8), D-E(9)

✓ MST tamamlandı! 4 kenar, 5 düğüm

Prim — Pseudocode & Priority Queue

Implementasyon detayları

Prim(graph, start):

```
in_mst = set()
key = {v: ∞ for v in graph}
parent = {v: None for v in graph}
key[start] = 0
pq = PriorityQueue()
pq.push((0, start))

while pq not empty:
    cost, u = pq.pop()
    if u in in_mst: continue
    in_mst.add(u)
    for v, w in graph[u].neighbors:
        if v not in in_mst and w < key[v]:
            key[v] = w
            parent[v] = u
            pq.push((w, v))
return parent # MST kenarları
```

key[v]

v'nin MST'ye bağlanma maliyeti. Güncellenir.

parent[v]

MST'de v'nin hangi düğüm üzerinden eklendiği

Priority Queue

Her adımda en küçük key değerini ver

in_mst kontrolü

Aynı düğümü iki kez işlemeyi önler

Zaman: $O(E \log V)$ — priority queue ile | Alan: $O(V)$

09

Kruskal Algoritması

Minimum Spanning Tree (MST)

Kruskal — Temel Fikir

Edge listesini sırala, döngü oluşturmadan ekle

1

Tüm kenarları ağırlığa göre küçükten büyüğe sırala

2

Her edge için: Eğer bu edge döngü oluşturmuyorsa MST'ye ekle

3

MST'de V-1 kenar olduğunda dur

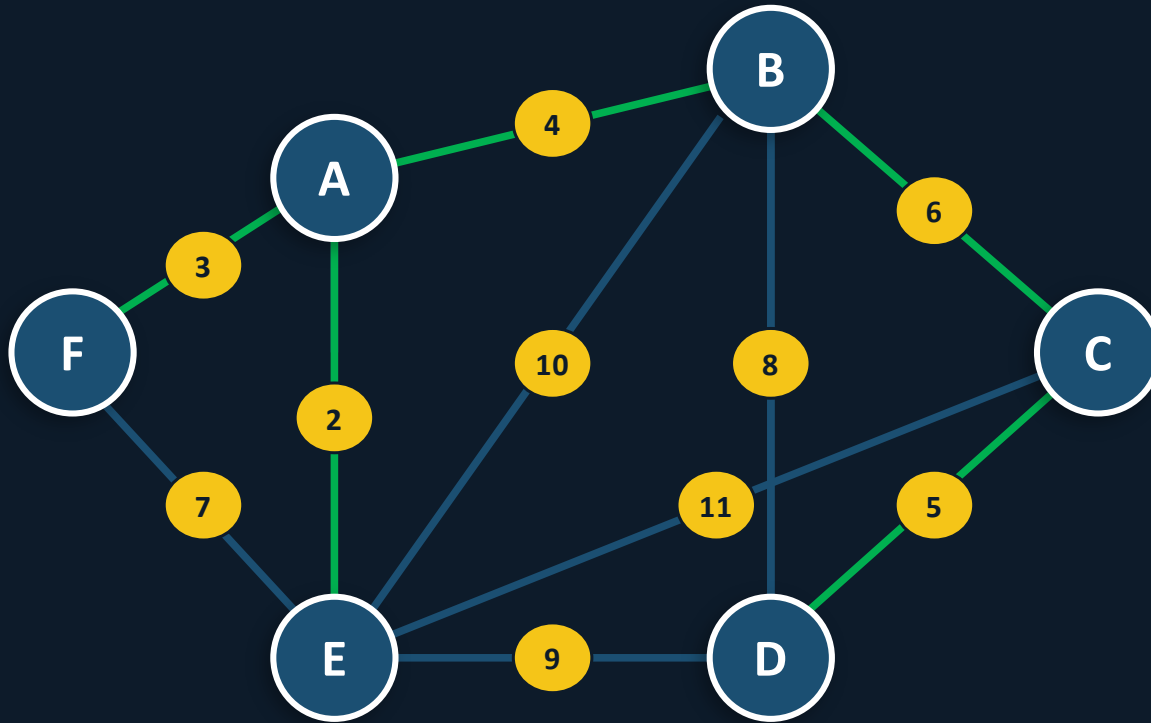
Union-Find (Disjoint Set Union) Veri Yapısı:

Kruskal, döngü tespiti için Union-Find kullanır. Her düğüm başta kendi kümesindedir. Bir kenar eklendiğinde iki küme birleştirilir. Eğer iki düğüm zaten aynı kümedeyse → döngü oluşur → bu kenarı atla.

Zaman Karmaşıklığı: $O(E \log E)$ — Kenarları sıralama baskın

Kruskal — Örnek Graph

Adım adım MST oluştur

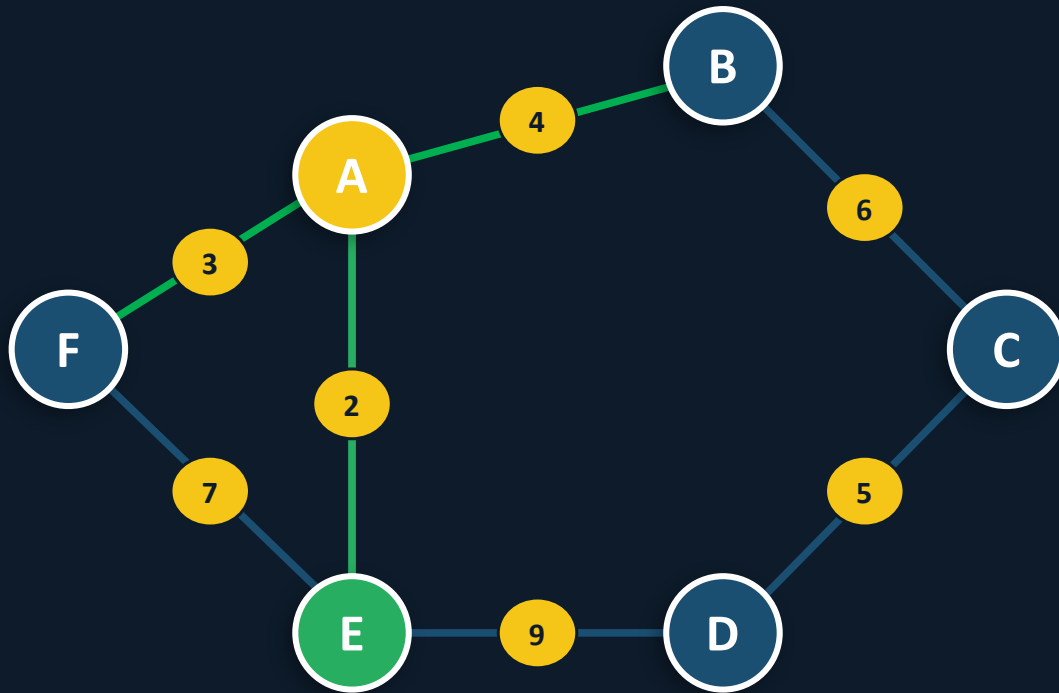


Kenarlar (Sıralı):

(A, E) → w=2	✓
(A, F) → w=3	✓
(A, B) → w=4	✓
(C, D) → w=5	✓
(B, C) → w=6	✓
(E, F) → w=7	
(B, D) → w=8	
(D, E) → w=9	
(B, E) → w=10	
(C, E) → w=11	

Kruskal — Adım Adım (1/2)

İlk 3 kenarı ekle



Mevcut MST: {A,E}, {A,F}, {A,B} → 3/5 kenar tamamlandı

Adım 1: (A,E) $w=2$

A ve E farklı kümelerde → EKLE
Kümeler: {A,E}, {B}, {C}, {D}, {F}

✓ MST'ye eklendi

Adım 2: (A,F) $w=3$

A ve F farklı kümelerde → EKLE
Kümeler: {A,E,F}, {B}, {C}, {D}

✓ MST'ye eklendi

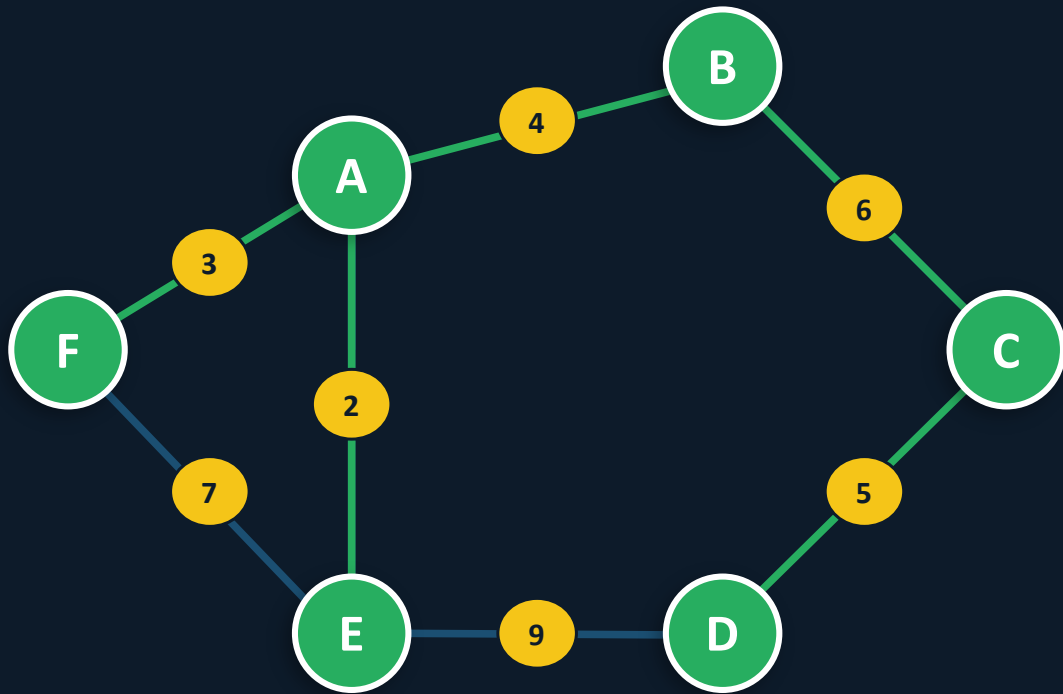
Adım 3: (A,B) $w=4$

A ve B farklı kümelerde → EKLE
Kümeler: {A,B,E,F}, {C}, {D}

✓ MST'ye eklendi

Kruskal — Adım Adım (2/2)

Döngü atla, MST'yi tamamla



Adım 4: (C,D) $w=5$

C ve D farklı kümelerde $\rightarrow$ EKLE
Kümeler: {A,B,E,F}, {C,D}

✓ MST'ye eklendi

Adım 5: (B,C) $w=6$

B ve C farklı kümelerde $\rightarrow$ EKLE
Kümeler: {A,B,C,D,E,F} — HEPSİ BİRLEŞTİ!

✓ MST tamamlandı! (5 kenar)

Adım 6: (E,F) $w=7$ — ATLA

E ve F zaten {A,B,C,D,E,F} kümesinde
Bu kenarı eklersek DÖNGÜ oluşur!

✗ Döngü — atla

MST Toplam Ağırlık: $2+3+4+5+6 = 20$ (6 düğüm, 5 kenar)

Kruskal — Pseudocode & Union-Find

Implementasyon detayları

Kruskal(graph):

```
edges = sorted(graph.edges, key=weight)
uf = UnionFind(graph.vertices)
mst = []

for (u, v, w) in edges:
    if uf.find(u) != uf.find(v):
        mst.append((u, v, w))
        uf.union(u, v)
    if len(mst) == |V| - 1:
        break

return mst
```

Union-Find

find(x):

Elemanın kümesini bul
(path compression)

union(x,y):

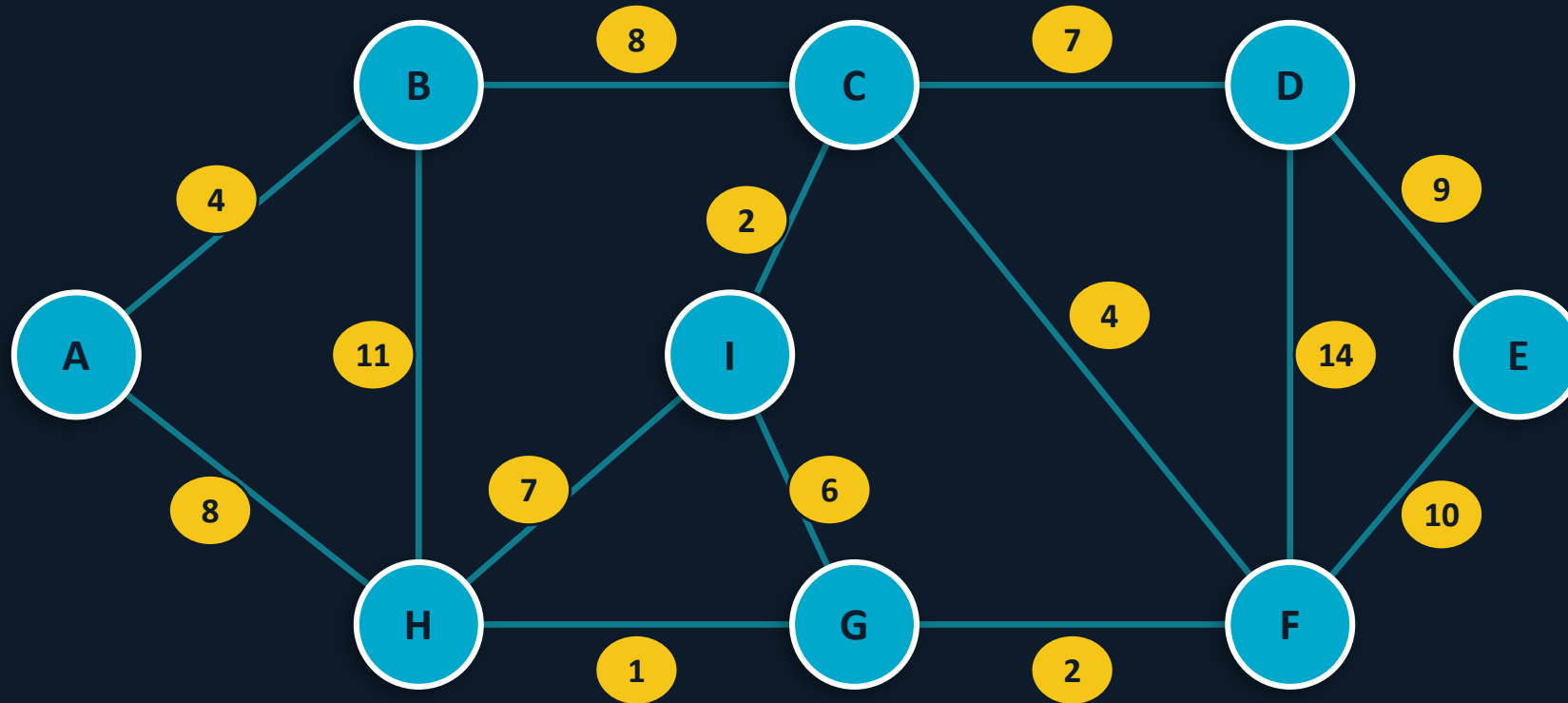
İki kümeyi birleştir
(union by rank)

Her ikisi $O(\alpha(n)) \approx O(1)$
(α = Ackermann inverse)

Kruskal vs Prim: Kruskal seyrek graphlar için, Prim yoğun graphlar için daha iyidir

Örnek Graph-Prim

Prim & Kruskal algoritmaları için — Yönsüz Ağırlıklı Graph



9 düğüm · 14 kenar · Yönsüz Ağırlıklı

Örnek Graph-Kruskal

Prim & Kruskal algoritmaları için — Yönsüz Ağırlıklı Graph

Kenar Listesi

H-G:1

C-I:2

G-F:2

A-B:4

C-F:4

I-G:6

C-D:7

I-H:7

A-H:8

B-C:8

D-E:9

F-E:10

B-H:11

D-F:14

9 düğüm · 14 kenar · Yönsüz Ağırlıklı

MST — Gerçek Hayat Uygulamaları

Kruskal nerede kullanılır?

Elektrik Ağı Tasarımı

Birden fazla şehri veya binayı elektrik ağına bağlarken minimum kablo kullan. MST, kablo maliyetini minimuma indirir.

💡 10 şehir ağı için: tüm bağlantılar 45 km → MST ile 9 bağlantı yeterli!

Ağ Altyapısı

Bilgisayar ağlarında tüm cihazları bağlayan minimum bandwidth tüketimli ağaç yapısı.

💡 *Spanning tree protokolü (STP) ağ döngülerini önler*

Yol / Boru Hattı

Birden fazla noktayı birbirine bağlayan minimum uzunlukta yol veya boru hattı tasarımı.

💡 *Su dağıtım ağı, doğalgaz hattı optimizasyonu*

Kümeleme (Clustering)

Veri madenciliğinde benzer noktaları gruplandırmak için MST kullanılır.

💡 *K-MST clustering: En ağır K-1 kenarı sil → K küme elde et*

Dijkstra vs Prim vs Kruskal



Dijkstra=En kısa yol | Prim=Yoğun MST | Kruskal=Seyrek MST

Üç greedy algoritmanın karşılaştırması

Özellik	Dijkstra	Prim	Kruskal
Amaç	En Kısa Yol	Min. Spanning Tree	Min. Spanning Tree
Başlangıç	Kaynak düğüm	Herhangi bir düğüm	Kenar listesi
Büyüme Şekli	Mesafeye göre	MST'ye komşu kesimden	Global en küçük kenar
Negatif Kenar	✗ Desteklemez	✓ Destekler	✓ Destekler
Veri Yapısı	Priority Queue	Priority Queue	Edge List + Union-Find
Zaman	$O((V+E) \log V)$	$O(E \log V)$	$O(E \log E)$
Yoğun Graph	Orta	✓ Daha iyi	✗ Yavaş
Seyrek Graph	✓ İyi	Orta	✓ Daha iyi
Sonuç	Kaynak → tüm yollar	MST kenar ağacı	MST kenar listesi

Algoritma Karmaşıklıkları — Özet

Tüm algoritmaların hızlı özeti

Algoritma	Zaman	Alan	Kullanım
BFS	$O(V + E)$	$O(V)$	En kısa yol (ağırlıksız), seviye analizi
DFS	$O(V + E)$	$O(V)$	Topo. sort, döngü tespiti, labirent
Dijkstra	$O((V+E) \log V)$	$O(V)$	En kısa yol (ağırlıklı, pozitif)
Kruskal	$O(E \log E)$	$O(V+E)$	Minimum Spanning Tree
Adj. Matrix	$O(V^2)$ bellek	$O(V^2)$	Yoğun graphlar, hızlı kenar sorgusu
Adj. List	$O(V+E)$ bellek	$O(V+E)$	Seyrek graphlar, komşulara erişim

V = düğüm sayısı | E = kenar sayısı

Özet — Neler Öğrendik?

Bu dersin kısa özeti



Graph Nedir?

$G=(V,E)$ — düğümler ve kenarlar, gerçek dünya problemlerini modeller



Graph Türleri

Directed/Undirected, Weighted, Connected, Complete, Cyclic/Acyclic



Temsil Yöntemleri

Adj. Matrix $O(V^2)$ · Adj. List $O(V+E)$ · Edge List $O(E)$



Dijkstra

Ağırlıklı en kısa yol, greedy, priority queue $O((V+E)\log V)$



Terminoloji

Vertex, edge, degree, in/out-degree, komşuluk ilişkileri



Yürüyüş Kavramları

Walk $\supseteq$ Trail $\supseteq$ Path, Cycle, Euler Trail/Circuit, Hamiltonian



BFS & DFS

BFS=Queue(en kısa yol) · DFS=Stack(döngü, topoloji) $O(V+E)$



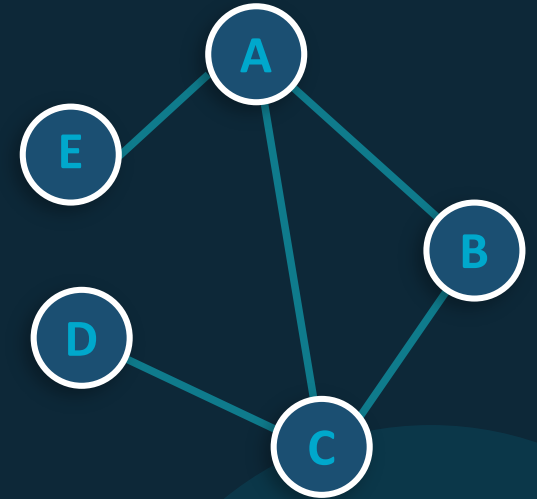
Kruskal

MST için greedy, Union-Find ile döngü tespiti $O(E \log E)$

Teşekkürler!

Veri Yapıları — Graph

BFS · DFS · Dijkstra · Prims Kruskal



Graph teorisini anlamak, gerçek dünya problemlerini çözmenin anahtarıdır.